

BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding

Jacob Devlin Ming-Wei Chang Kenton Lee Kristina Toutanova

Google AI Language

{jacobdevlin, mingweichang, kentonl, kristout}@google.com

Abstract

We introduce a new language representation model called **BERT**, which stands for **Bidirectional Encoder Representations from Transformers**. Unlike recent language representation models (Peters et al., 2018a; Radford et al., 2018), BERT is designed to pre-train deep bidirectional representations from unlabeled text by jointly conditioning on both left and right context in all layers. As a result, the pre-trained BERT model can be fine-tuned with just one additional output layer to create state-of-the-art models for a wide range of tasks, such as question answering and language inference, without substantial task-specific architecture modifications.

BERT is conceptually simple and empirically powerful. It obtains new state-of-the-art results on eleven natural language processing tasks, including pushing the GLUE score to 80.5% (7.7% point absolute improvement), MultiNLI accuracy to 86.7% (4.6% absolute improvement), SQuAD v1.1 question answering Test F1 to 93.2 (1.5 point absolute improvement) and SQuAD v2.0 Test F1 to 83.1 (5.1 point absolute improvement).

1 Introduction

Language model pre-training has been shown to be effective for improving many natural language processing tasks (Dai and Le, 2015; Peters et al., 2018a; Radford et al., 2018; Howard and Ruder, 2018). These include sentence-level tasks such as natural language inference (Bowman et al., 2015; Williams et al., 2018) and paraphrasing (Dolan and Brockett, 2005), which aim to predict the relationships between sentences by analyzing them holistically, as well as token-level tasks such as named entity recognition and question answering, where models are required to produce fine-grained output at the token level (Tjong Kim Sang and De Meulder, 2003; Rajpurkar et al., 2016).

There are two existing strategies for applying pre-trained language representations to downstream tasks: *feature-based* and *fine-tuning*. The feature-based approach, such as ELMo (Peters et al., 2018a), uses task-specific architectures that include the pre-trained representations as additional features. The fine-tuning approach, such as the Generative Pre-trained Transformer (OpenAI GPT) (Radford et al., 2018), introduces minimal task-specific parameters, and is trained on the downstream tasks by simply fine-tuning *all* pre-trained parameters. The two approaches share the same objective function during pre-training, where they use unidirectional language models to learn general language representations.

We argue that current techniques restrict the power of the pre-trained representations, especially for the fine-tuning approaches. The major limitation is that standard language models are unidirectional, and this limits the choice of architectures that can be used during pre-training. For example, in OpenAI GPT, the authors use a left-to-right architecture, where every token can only attend to previous tokens in the self-attention layers of the Transformer (Vaswani et al., 2017). Such restrictions are sub-optimal for sentence-level tasks, and could be very harmful when applying fine-tuning based approaches to token-level tasks such as question answering, where it is crucial to incorporate context from both directions.

In this paper, we improve the fine-tuning based approaches by proposing BERT: **Bidirectional Encoder Representations from Transformers**. BERT alleviates the previously mentioned unidirectionality constraint by using a “masked language model” (MLM) pre-training objective, inspired by the Cloze task (Taylor, 1953). The masked language model randomly masks some of the tokens from the input, and the objective is to predict the original vocabulary id of the masked

BERT: 面向语言理解的深度双向 Transformer 预训练

Jacob Devlin Ming-Wei Chang Kenton Lee Kristina Toutanova

Google AI 语言团队

{jacobdevlin, mingweichang, kentonl, kristout}@google.com

摘要

我们提出一种新的语言表征模型，称为 BERT（来自 Transformer 的双向编码器表征）。与近期语言表征模型（Peters 等，2018a；Radford 等，2018）不同，BERT 专为通过在所有层中联合依赖左右上下文，从无标签文本中预训练深度双向表征而设计。因此，预训练后的 BERT 模型仅需添加一个额外输出层即可微调，从而为问答、语言推理等广泛任务构建技术前沿模型，且无需大幅修改任务专用架构。

将预训练语言表征应用于下游任务目前存在两种策略：基于特征的方法和微调方法。基于特征的方法（如 ELMo，Peters 等，2018a）采用任务专用架构，将预训练表征作为附加特征纳入其中。微调方法（如生成式预训练 Transformer（OpenAI GPT），Radford 等，2018）仅引入极少的任务专用参数，并通过简单地微调全部预训练参数，在下游任务上完成训练。这两种方法在预训练阶段共享相同的目标函数，即均采用单向语言模型学习通用语言表征。

BERT 在概念上简洁明了，实证效果强大。它在 11 项自然语言处理任务上刷新了技术前沿，包括将 GLUE 分数推升至 80.5%（绝对提升 7.7 个百分点）、MultiNLI 准确率提升至 86.7%（绝对提升 4.6 个百分点）、SQuAD v1.1 问答测试集 F1 值达 93.2（绝对提升 1.5 个百分点），以及 SQuAD v2.0 测试集 F1 值达 83.1（绝对提升 5.1 个百分点）。

我们认为，当前技术限制了预训练表征的能力，尤其体现在微调方法上。主要局限在于标准语言模型为单向结构，这限制了预训练阶段可采用的架构类型。例如，在 OpenAI GPT 中，作者采用了从左到右的架构，其中每个词元在 Transformer 的自注意力层中仅能关注其前面的词元（Vaswani 等，2017）。此类限制对于句子级任务而言次优，而在将基于微调的方法应用于问答等词元级任务时可能尤为有害，因为后者必须充分融合来自两个方向的上下文信息。

1 引言

语言模型预训练已被证实可有效提升众多自然语言处理任务的性能（Dai 和 Le，2015；Peters 等，2018a；Radford 等，2018；Howard 和 Ruder，2018）。这些任务包括句子级任务，例如自然语言推理（Bowman 等，2015；Williams 等，2018）和复述（Dolan 和 Brockett，2005），旨在通过整体分析句子来预测句子间关系；还包括词元级任务，例如命名实体识别和问答，要求模型在词元级别输出细粒度结果（Tjong Kim Sang 和 De Meulder，2003；Rajpurkar 等，2016）。

本文中，我们通过提出 BERT（来自 Transformer 的双向编码器表征）改进了基于微调的方法。BERT 借助受完形填空任务（Taylor，1953）启发的掩码语言建模（MLM）预训练目标，缓解了前述单向性约束。掩码语言建模随机遮盖输入中部分词元，其目标是预测被遮盖词元原本的词汇表 ID。

word based only on its context. Unlike left-to-right language model pre-training, the MLM objective enables the representation to fuse the left and the right context, which allows us to pre-train a deep bidirectional Transformer. In addition to the masked language model, we also use a “next sentence prediction” task that jointly pre-trains text-pair representations. The contributions of our paper are as follows:

- We demonstrate the importance of bidirectional pre-training for language representations. Unlike Radford et al. (2018), which uses unidirectional language models for pre-training, BERT uses masked language models to enable pre-trained deep bidirectional representations. This is also in contrast to Peters et al. (2018a), which uses a shallow concatenation of independently trained left-to-right and right-to-left LMs.
- We show that pre-trained representations reduce the need for many heavily-engineered task-specific architectures. BERT is the first fine-tuning based representation model that achieves state-of-the-art performance on a large suite of sentence-level *and* token-level tasks, outperforming many task-specific architectures.
- BERT advances the state of the art for eleven NLP tasks. The code and pre-trained models are available at <https://github.com/google-research/bert>.

2 Related Work

There is a long history of pre-training general language representations, and we briefly review the most widely-used approaches in this section.

2.1 Unsupervised Feature-based Approaches

Learning widely applicable representations of words has been an active area of research for decades, including non-neural (Brown et al., 1992; Ando and Zhang, 2005; Blitzer et al., 2006) and neural (Mikolov et al., 2013; Pennington et al., 2014) methods. Pre-trained word embeddings are an integral part of modern NLP systems, offering significant improvements over embeddings learned from scratch (Turian et al., 2010). To pre-train word embedding vectors, left-to-right language modeling objectives have been used (Mnih and Hinton, 2009), as well as objectives to discriminate correct from incorrect words in left and right context (Mikolov et al., 2013).

These approaches have been generalized to coarser granularities, such as sentence embeddings (Kiros et al., 2015; Logeswaran and Lee, 2018) or paragraph embeddings (Le and Mikolov, 2014). To train sentence representations, prior work has used objectives to rank candidate next sentences (Jernite et al., 2017; Logeswaran and Lee, 2018), left-to-right generation of next sentence words given a representation of the previous sentence (Kiros et al., 2015), or denoising auto-encoder derived objectives (Hill et al., 2016).

ELMo and its predecessor (Peters et al., 2017, 2018a) generalize traditional word embedding research along a different dimension. They extract *context-sensitive* features from a left-to-right and a right-to-left language model. The contextual representation of each token is the concatenation of the left-to-right and right-to-left representations. When integrating contextual word embeddings with existing task-specific architectures, ELMo advances the state of the art for several major NLP benchmarks (Peters et al., 2018a) including question answering (Rajpurkar et al., 2016), sentiment analysis (Socher et al., 2013), and named entity recognition (Tjong Kim Sang and De Meulder, 2003). Melamud et al. (2016) proposed learning contextual representations through a task to predict a single word from both left and right context using LSTMs. Similar to ELMo, their model is feature-based and not deeply bidirectional. Fedus et al. (2018) shows that the cloze task can be used to improve the robustness of text generation models.

2.2 Unsupervised Fine-tuning Approaches

As with the feature-based approaches, the first works in this direction only pre-trained word embedding parameters from unlabeled text (Collobert and Weston, 2008).

More recently, sentence or document encoders which produce contextual token representations have been pre-trained from unlabeled text and fine-tuned for a supervised downstream task (Dai and Le, 2015; Howard and Ruder, 2018; Radford et al., 2018). The advantage of these approaches is that few parameters need to be learned from scratch. At least partly due to this advantage, OpenAI GPT (Radford et al., 2018) achieved previously state-of-the-art results on many sentence-level tasks from the GLUE benchmark (Wang et al., 2018a). Left-to-right language model-

仅依据其上下文预测词。与从左到右的语言模型预训练不同，掩码语言建模 (MLM) 目标使表征能够融合左右上下文，从而支持我们预训练深度双向 Transformer。除掩码语言建模外，我们还引入下一句预测任务，联合预训练文本对表征。本文贡献如下：

- 我们验证了双向预训练对语言表征的重要性。不同于 Radford 等 (2018) 采用单向语言模型进行预训练，BERT 使用掩码语言模型实现深度双向表征的预训练。这也区别于 Peters 等 (2018a) 所采用的浅层拼接方式，即分别独立训练从左到右和从右到左的语言模型。
- 我们表明，预训练表征降低了对大量高度定制化任务专用架构的需求。BERT 是首个基于微调的表征模型，在一系列句子级和词元级任务上均达到技术前沿水平，其表现优于众多任务专用架构。
- BERT 在 11 项自然语言处理任务上刷新了技术前沿。代码与预训练模型可在 <https://github.com/google-research/bert> 获取。

2 相关工作

预训练通用语言表征已有悠久历史，本节简要回顾当前最广泛应用的几类方法。

2.1 无监督基于特征的方法

学习通用性强的词表征已有数十年的研究历史，包括非神经网络方法 (Brown 等, 1992; Ando 和 Zhang, 2005; Blitzer 等, 2006) 和神经网络方法 (Mikolov 等, 2013; Pennington 等, 2014)。预训练词嵌入已成为现代自然语言处理系统不可或缺的组成部分，相较于从零开始学习的嵌入，性能显著提升 (Turian 等, 2010)。为预训练词嵌入向量，曾使用过从左到右的语言建模目标 (Mnih 和 Hinton, 2009)，也使用过判别左右上下文中正确词与错误词的目标 (Mikolov 等, 2013)。

这些方法已被推广至更粗粒度的表示，例如句子嵌入 (Kiros 等, 2015; Logeswaran 和 Lee, 2018) 或段落嵌入 (Le 和 Mikolov, 2014)。为训练句子表征，先前研究采用过多种目标函数：对候选下一句进行排序 (Jernite 等, 2017; Logeswaran 和 Lee, 2018)、根据前一句的表征从左到右生成下一句的词 (Kiros 等, 2015)，或基于去噪自编码器的目标函数 (Hill 等, 2016)。

ELMo 及其前身 (Peters 等, 2017, 2018a) 沿不同维度拓展了传统词嵌入研究。它们分别从一个从左到右和一个从右到左的语言模型中提取上下文敏感特征。每个词元的上下文表征是这两个方向表征的拼接。当将上下文词嵌入整合进现有任务专用架构时，ELMo 在若干主流自然语言处理基准测试 (Peters 等, 2018a) 上推动了技术前沿，包括问答 (Rajpurkar 等, 2016)、情感分析 (Socher 等, 2013) 以及命名实体识别 (Tjong Kim Sang 和 De Meulder, 2003)。Melamud 等 (2016) 提出通过一项任务来学习上下文表征：利用 LSTM 预测同时处于左右上下文中的单个词。与 ELMo 类似，他们的模型属于基于特征的方法，且并非深度双向结构。Fedus 等 (2018) 表明，完形填空任务可用于提升文本生成模型的鲁棒性。

2.2 无监督微调方法

与基于特征的方法类似，该方向的早期工作仅从无标签文本中预训练词嵌入参数 (Collobert 和 Weston, 2008)。

近年来，人们已开始对能生成上下文相关词元表征的句子或文档编码器进行无监督预训练，并在有监督下游任务中进行微调 (Dai 和 Le, 2015; Howard 和 Ruder, 2018; Radford 等, 2018)。这类方法的优势在于，仅需从头学习少量参数。至少部分得益于这一优势，OpenAI GPT (Radford 等, 2018) 在 GLUE 基准测试 (Wang 等, 2018a) 中的多个句子级任务上取得了此前最先进的结果。从左到右的语言模型——

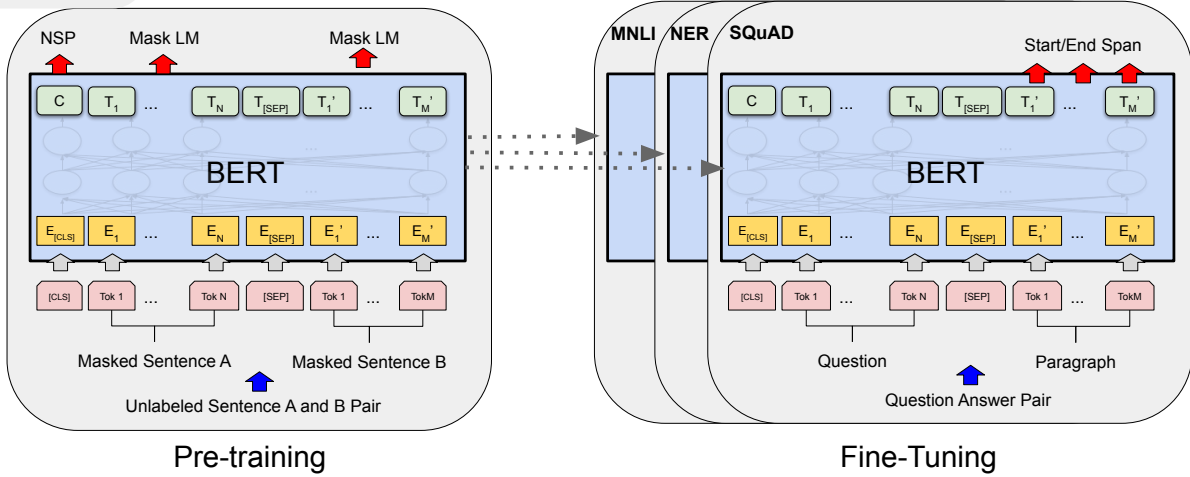


Figure 1: Overall pre-training and fine-tuning procedures for BERT. Apart from output layers, the same architectures are used in both pre-training and fine-tuning. The same pre-trained model parameters are used to initialize models for different down-stream tasks. During fine-tuning, all parameters are fine-tuned. [CLS] is a special symbol added in front of every input example, and [SEP] is a special separator token (e.g. separating questions/answers).

ing and auto-encoder objectives have been used for pre-training such models (Howard and Ruder, 2018; Radford et al., 2018; Dai and Le, 2015).

2.3 Transfer Learning from Supervised Data

There has also been work showing effective transfer from supervised tasks with large datasets, such as natural language inference (Conneau et al., 2017) and machine translation (McCann et al., 2017). Computer vision research has also demonstrated the importance of transfer learning from large pre-trained models, where an effective recipe is to fine-tune models pre-trained with ImageNet (Deng et al., 2009; Yosinski et al., 2014).

3 BERT

We introduce BERT and its detailed implementation in this section. There are two steps in our framework: *pre-training* and *fine-tuning*. During pre-training, the model is trained on unlabeled data over different pre-training tasks. For fine-tuning, the BERT model is first initialized with the pre-trained parameters, and all of the parameters are fine-tuned using labeled data from the downstream tasks. Each downstream task has separate fine-tuned models, even though they are initialized with the same pre-trained parameters. The question-answering example in Figure 1 will serve as a running example for this section.

A distinctive feature of BERT is its unified architecture across different tasks. There is mini-

mal difference between the pre-trained architecture and the final downstream architecture.

Model Architecture BERT’s model architecture is a multi-layer bidirectional Transformer encoder based on the original implementation described in Vaswani et al. (2017) and released in the `tensorflow/tensor2tensor` library.¹ Because the use of Transformers has become common and our implementation is almost identical to the original, we will omit an exhaustive background description of the model architecture and refer readers to Vaswani et al. (2017) as well as excellent guides such as “The Annotated Transformer.”²

In this work, we denote the number of layers (i.e., Transformer blocks) as L , the hidden size as H , and the number of self-attention heads as A .³ We primarily report results on two model sizes: **BERT_{BASE}** ($L=12$, $H=768$, $A=12$, Total Parameters=110M) and **BERT_{LARGE}** ($L=24$, $H=1024$, $A=16$, Total Parameters=340M).

BERT_{BASE} was chosen to have the same model size as OpenAI GPT for comparison purposes. Critically, however, the BERT Transformer uses bidirectional self-attention, while the GPT Transformer uses constrained self-attention where every token can only attend to context to its left.⁴

¹<https://github.com/tensorflow/tensor2tensor>

²<http://nlp.seas.harvard.edu/2018/04/03/attention.html>

³In all cases we set the feed-forward/filter size to be $4H$, i.e., 3072 for the $H = 768$ and 4096 for the $H = 1024$.

⁴We note that in the literature the bidirectional Trans-

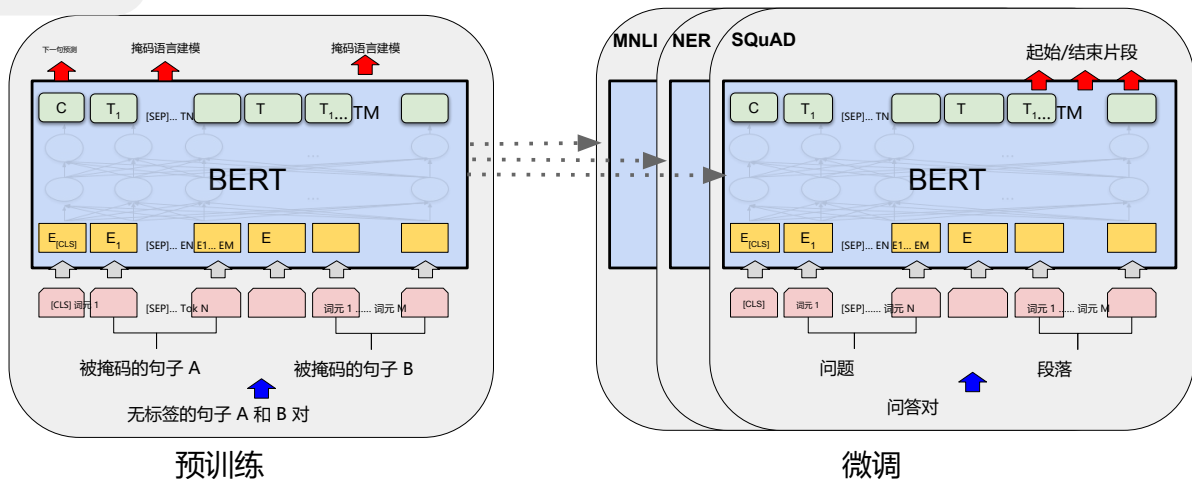


图 1: BERT 的整体预训练与微调流程。除输出层外, 预训练与微调均采用相同架构。同一预训练模型参数用于初始化不同下游任务的模型。在微调阶段, 所有参数均参与微调。[CLS] 是添加在每个输入样例最前端的特殊符号, [SEP] 是特殊分隔符 (例如用于分隔问题与答案)。

及自编码器目标已被用于此类模型的预训练 (Howard 和 Ruder, 2018; Radford 等, 2018; Dai 和 Le, 2015)。

2.3 来自有标签数据的迁移学习

亦有研究工作表明, 从大规模有标签数据集上的监督任务迁移学习同样有效, 例如自然语言推理 (Conneau 等, 2017) 与机器翻译 (McCann 等, 2017)。计算机视觉领域的研究同样证实了从大规模预训练模型迁移学习的重要性, 其中一种有效策略是对在 ImageNet (Deng 等, 2009; Yosinski 等, 2014) 上预训练的模型进行微调。

3 BERT

本节介绍 BERT 及其详细实现。我们的框架包含两个步骤: 预训练与微调。在预训练阶段, 模型在大量无标签数据上, 通过不同的预训练任务进行训练。在微调阶段, BERT 模型首先以预训练所得参数进行初始化, 再利用下游任务的有标签数据对全部参数进行微调。每个下游任务均有各自独立的微调模型, 即使它们均以相同的预训练参数初始化。图 1 中的问答示例将作为本节的贯穿实例。

BERT 的一个显著特点是其跨不同任务的统一架构。其间仅有极小的

预训练架构与最终下游架构之间的显著差异。模型架构 BERT 的模型架构基于 Vaswani 等

(2017) 提出的原始实现, 并在 tensor2tensor 库中发布。¹ 由于 Transformer 的应用已十分普遍, 且我们的实现与其原始版本几乎完全相同, 因此我们将省略对该模型架构的详尽背景介绍, 并建议读者参阅 Vaswani 等 (2017) 以及《带注释的 Transformer》等优秀指南。²

本文中, 我们将层数 (即 Transformer 块数) 记为 L , 隐藏层维度记为 H , 自注意力头数记为 A 。³ 我们主要报告两类模型规模的结果: BERTBASE ($L=12$, $H=768$, $A=12$, 总参数量=1.1 亿) 与 BERTLARGE ($L=24$, $H=1024$, $A=16$, 总参数量=3.4 亿)。

为便于比较, BERTBASE 的模型规模被设定为与 OpenAI GPT 相同。然而关键区别在于: BERT 的 Transformer 使用双向自注意力机制, 而 GPT 的 Transformer 使用受限自注意力机制, 其中每个词元仅能关注其左侧的上下文。⁴

¹<https://github.com/tensorflow/tensor2tensor>

²<http://nlp.seas.harvard.edu/2018/04/03/attention.html>

³ 在所有情况下, 我们将前馈网络/滤波器尺寸设为 $4H$, 即当 $H=768$ 时为 3072, 当 $H=1024$ 时为 4096。

⁴ 我们注意到, 文献中双向 Transformer 的

Input/Output Representations To make BERT handle a variety of down-stream tasks, our input representation is able to unambiguously represent both a single sentence and a pair of sentences (e.g., $\langle \text{Question, Answer} \rangle$) in one token sequence. Throughout this work, a “sentence” can be an arbitrary span of contiguous text, rather than an actual linguistic sentence. A “sequence” refers to the input token sequence to BERT, which may be a single sentence or two sentences packed together.

We use WordPiece embeddings (Wu et al., 2016) with a 30,000 token vocabulary. The first token of every sequence is always a special classification token ($[\text{CLS}]$). The final hidden state corresponding to this token is used as the aggregate sequence representation for classification tasks. Sentence pairs are packed together into a single sequence. We differentiate the sentences in two ways. First, we separate them with a special token ($[\text{SEP}]$). Second, we add a learned embedding to every token indicating whether it belongs to sentence A or sentence B. As shown in Figure 1, we denote input embedding as E , the final hidden vector of the special $[\text{CLS}]$ token as $C \in \mathbb{R}^H$, and the final hidden vector for the i^{th} input token as $T_i \in \mathbb{R}^H$.

For a given token, its input representation is constructed by summing the corresponding token, segment, and position embeddings. A visualization of this construction can be seen in Figure 2.

3.1 Pre-training BERT

Unlike Peters et al. (2018a) and Radford et al. (2018), we do not use traditional left-to-right or right-to-left language models to pre-train BERT. Instead, we pre-train BERT using two unsupervised tasks, described in this section. This step is presented in the left part of Figure 1.

Task #1: Masked LM Intuitively, it is reasonable to believe that a deep bidirectional model is strictly more powerful than either a left-to-right model or the shallow concatenation of a left-to-right and a right-to-left model. Unfortunately, standard conditional language models can only be trained left-to-right *or* right-to-left, since bidirectional conditioning would allow each word to indirectly “see itself”, and the model could trivially predict the target word in a multi-layered context.

former is often referred to as a “Transformer encoder” while the left-context-only version is referred to as a “Transformer decoder” since it can be used for text generation.

In order to train a deep bidirectional representation, we simply mask some percentage of the input tokens at random, and then predict those masked tokens. We refer to this procedure as a “masked LM” (MLM), although it is often referred to as a *Cloze* task in the literature (Taylor, 1953). In this case, the final hidden vectors corresponding to the mask tokens are fed into an output softmax over the vocabulary, as in a standard LM. In all of our experiments, we mask 15% of all WordPiece tokens in each sequence at random. In contrast to denoising auto-encoders (Vincent et al., 2008), we only predict the masked words rather than reconstructing the entire input.

Although this allows us to obtain a bidirectional pre-trained model, a downside is that we are creating a mismatch between pre-training and fine-tuning, since the $[\text{MASK}]$ token does not appear during fine-tuning. To mitigate this, we do not always replace “masked” words with the actual $[\text{MASK}]$ token. The training data generator chooses 15% of the token positions at random for prediction. If the i -th token is chosen, we replace the i -th token with (1) the $[\text{MASK}]$ token 80% of the time (2) a random token 10% of the time (3) the unchanged i -th token 10% of the time. Then, T_i will be used to predict the original token with cross entropy loss. We compare variations of this procedure in Appendix C.2.

Task #2: Next Sentence Prediction (NSP)

Many important downstream tasks such as Question Answering (QA) and Natural Language Inference (NLI) are based on understanding the *relationship* between two sentences, which is not directly captured by language modeling. In order to train a model that understands sentence relationships, we pre-train for a binarized *next sentence prediction* task that can be trivially generated from any monolingual corpus. Specifically, when choosing the sentences A and B for each pre-training example, 50% of the time B is the actual next sentence that follows A (labeled as IsNext), and 50% of the time it is a random sentence from the corpus (labeled as NotNext). As we show in Figure 1, C is used for next sentence prediction (NSP).⁵ Despite its simplicity, we demonstrate in Section 5.1 that pre-training towards this task is very beneficial to both QA and NLI.⁶

⁵The final model achieves 97%-98% accuracy on NSP.

⁶The vector C is not a meaningful sentence representation without fine-tuning, since it was trained with NSP.

输入/输出表征 为使 BERT 能处理多种下游任务，我们的输入表征可明确无歧义地表示单一句子或句对（例如〈问题，答案〉）的单一词元序列。在本文中，“句子”可指任意连续文本片段，而非严格的语言学意义上的句子；“序列”则指输入至 BERT 的词元序列，其可能为单个句子，也可能为打包在一起的两个句子。

我们采用 WordPiece 嵌入 (Wu 等, 2016)，词表大小为 30,000。每个序列的首个词元始终为一个特殊的分类标记 ([CLS])。对应于该标记的最终隐藏状态被用作分类任务的聚合序列表征。句对被打包为单一序列。我们通过两种方式区分句子：第一，用特殊标记 ([SEP]) 分隔；第二，为每个词元添加一个可学习的嵌入，以指示其属于句子 A 还是句子 B。如图 1 所示，我们将输入嵌入记为 E ，特殊 [CLS] 标记的最终隐藏向量记为 $C \in \mathbb{R}^H$ ，第 i 个输入词元的最终隐藏向量记为 $T_i \in \mathbb{R}^H$ 。

对于给定词元，其输入表征由对应的词元嵌入、片段嵌入与位置嵌入之和构成。该构造过程的可视化示意图 2。

3.1 BERT 预训练

与 Peters 等 (2018a) 及 Radford 等 (2018) 不同，我们并未采用传统的左到右或右到左语言模型来预训练 BERT。相反，我们采用本节所述的两项无监督任务对 BERT 进行预训练。该步骤如图 1 左侧所示。

任务 #1：掩码语言模型 (MLM) 直观来看，可以合理认为：深度双向模型严格强于左到右模型，也强于左到右模型与右到左模型的浅层拼接。遗憾的是，标准的条件语言模型只能按左到右或右到左顺序训练，因为双向条件会使每个词间接地‘看到’自身，从而使模型可在多层上下文中轻易预测目标词。

为训练深度双向表征，我们只需随机屏蔽输入词元中一定比例的词元，然后预测这些被屏蔽的词元。我们将该流程称为掩码语言模型 (MLM)，尽管文献中常称之为完形填空 (Cloze) 任务 (Taylor, 1953)。在此情形下，对应于掩码词元的最终隐藏向量将输入至词汇表上的输出 softmax 层，与标准语言模型一致。在我们的所有实验中，我们均随机屏蔽每个序列中 15% 的 WordPiece 词元。与去噪自编码器 (Vincent 等, 2008) 不同，我们仅预测被掩码的词，而非重构整个输入。

尽管这使我们能够获得一个双向预训练模型，但其缺点在于预训练与微调之间存在不匹配，因为 [MASK] 标记在微调阶段并不会出现。为缓解这一问题，我们并非总是用真实的 [MASK] 标记替换被掩码的词。训练数据生成器会随机选择 15% 的词元位置进行预测。若第 i 个词元被选中，则以如下方式替换该词元：(1) 80% 的概率替换为 [MASK] 标记；(2) 10% 的概率替换为一个随机词元；(3) 10% 的概率保持第 i 个词元不变。随后， T_i 将用于通过交叉熵损失预测原始词元。我们在附录 C.2 中对比了该流程的不同变体。

任务 #2：下一句预测 (NSP)

许多重要的下游任务（例如问答 (QA) 和自然语言推理 (NLI)）均依赖于对两个句子之间关系的理解，而这种关系无法直接通过语言建模来捕获。为训练一个能理解句子关系的模型，我们针对一个二分类的下一句预测任务进行预训练；该任务可直接从任意单语语料库中轻松生成。具体而言，在为每个预训练样本选择句子 A 和 B 时，50% 的情况下 B 是实际紧随 A 之后的句子（标记为 IsNext），其余 50% 的情况下 B 则是从语料库中随机选取的句子（标记为 NotNext）。如图 1 所示，C 用于下一句预测 (NSP)。5 尽管该任务十分简单，我们在第 5.1 节中仍证明：针对该任务的预训练对 QA 和 NLI 均大有裨益。6

前者通常被称为 Transformer 编码器，而仅使用左侧上下文的版本则被称为 Transformer 解码器，因其可用于文本生成。

5 最终模型在 NSP 任务上达到 97%–98% 的准确率。

6 向量 C 在未经微调的情况下并非有意义的句子表征，因其是通过下一句预测 (NSP) 任务训练得到的。

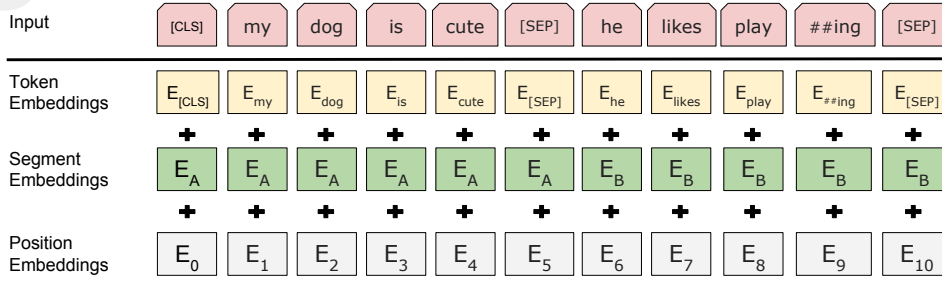


Figure 2: BERT input representation. The input embeddings are the sum of the token embeddings, the segmentation embeddings and the position embeddings.

The NSP task is closely related to representation-learning objectives used in [Jernite et al. \(2017\)](#) and [Logeswaran and Lee \(2018\)](#). However, in prior work, only sentence embeddings are transferred to down-stream tasks, where BERT transfers all parameters to initialize end-task model parameters.

Pre-training data The pre-training procedure largely follows the existing literature on language model pre-training. For the pre-training corpus we use the BooksCorpus (800M words) ([Zhu et al., 2015](#)) and English Wikipedia (2,500M words). For Wikipedia we extract only the text passages and ignore lists, tables, and headers. It is critical to use a document-level corpus rather than a shuffled sentence-level corpus such as the Billion Word Benchmark ([Chelba et al., 2013](#)) in order to extract long contiguous sequences.

3.2 Fine-tuning BERT

Fine-tuning is straightforward since the self-attention mechanism in the Transformer allows BERT to model many downstream tasks—whether they involve single text or text pairs—by swapping out the appropriate inputs and outputs. For applications involving text pairs, a common pattern is to independently encode text pairs before applying bidirectional cross attention, such as [Parikh et al. \(2016\)](#); [Seo et al. \(2017\)](#). BERT instead uses the self-attention mechanism to unify these two stages, as encoding a concatenated text pair with self-attention effectively includes *bidirectional* cross attention between two sentences.

For each task, we simply plug in the task-specific inputs and outputs into BERT and fine-tune all the parameters end-to-end. At the input, sentence A and sentence B from pre-training are analogous to (1) sentence pairs in paraphrasing, (2) hypothesis-premise pairs in entailment, (3) question-passage pairs in question answering, and

(4) a degenerate text- $\emptyset$ pair in text classification or sequence tagging. At the output, the token representations are fed into an output layer for token-level tasks, such as sequence tagging or question answering, and the $[CLS]$ representation is fed into an output layer for classification, such as entailment or sentiment analysis.

Compared to pre-training, fine-tuning is relatively inexpensive. All of the results in the paper can be replicated in at most 1 hour on a single Cloud TPU, or a few hours on a GPU, starting from the exact same pre-trained model.⁷ We describe the task-specific details in the corresponding subsections of Section 4. More details can be found in Appendix A.5.

4 Experiments

In this section, we present BERT fine-tuning results on 11 NLP tasks.

4.1 GLUE

The General Language Understanding Evaluation (GLUE) benchmark ([Wang et al., 2018a](#)) is a collection of diverse natural language understanding tasks. Detailed descriptions of GLUE datasets are included in Appendix B.1.

To fine-tune on GLUE, we represent the input sequence (for single sentence or sentence pairs) as described in Section 3, and use the final hidden vector $C \in \mathbb{R}^H$ corresponding to the first input token ($[CLS]$) as the aggregate representation. The only new parameters introduced during fine-tuning are classification layer weights $W \in \mathbb{R}^{K \times H}$, where K is the number of labels. We compute a standard classification loss with C and W , i.e., $\log(\text{softmax}(CW^T))$.

⁷For example, the BERT SQuAD model can be trained in around 30 minutes on a single Cloud TPU to achieve a Dev F1 score of 91.0%.

⁸See (10) in <https://gluebenchmark.com/faq>.

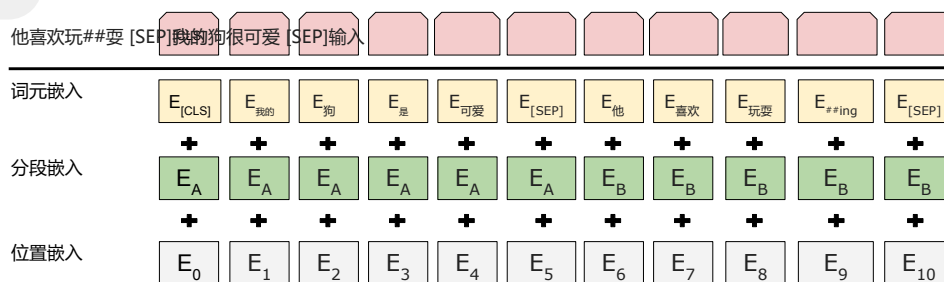


图2：BERT输入表征。输入嵌入为词元嵌入、分段嵌入与位置嵌入三者之和。

NSP 任务与 Jernite 等 (2017) 及 Logeswaran与Lee (2018) 中使用的表征学习目标密切相关。然而，在先前工作中，仅有句子嵌入被迁移到下游任务，而BERT则迁移全部参数以初始化下游任务模型的参数。预训练数据：预训练流程大体遵循现有语言模型预

训练文献。我们采用BookCorpus (8亿词) (Zhu等, 2015) 和英文维基百科 (25亿词) 作为预训练语料。对于维基百科，我们仅提取正文段落，忽略列表、表格和标题。使用文档级语料（而非如十亿词基准 (Chelba等, 2013) 这类经打乱句子级语料）至关重要，以便提取长而连续的序列。

3.2 BERT微调

微调过程十分直接，因为Transformer中的自注意力机制使BERT能够通过替换适当的输入与输出，建模众多下游任务——无论这些任务涉及单文本还是文本对。对于涉及文本对的应用，一种常见模式是先独立编码文本对，再施加双向交叉注意力（例如 Parikh 等, 2016; Seo等, 2017）。而BERT则利用自注意力机制将这两个阶段统一起来：对拼接后的文本对应用自注意力编码，本质上已包含了两个句子之间的双向交叉注意力。

针对每项任务，我们只需将该任务特定的输入和输出接入BERT，并端到端地微调全部参数。在输入端，预训练阶段所用的句子A与句子B分别对应于：（1）复述任务中的句子对，（2）蕴含任务中的假设-前提对，（3）问答任务中的问题-段落对，以及

（4）文本分类或序列标注任务中的退化文本-空对。在输出端，词元表征被送入输出层以处理词元级任务（例如序列标注或问答），而 [CLS] 表征则被送入输出层以处理分类任务（例如蕴含判断或情感分析）。

与预训练相比，微调的成本相对较低。论文中的所有实验结果均可在单个云TPU上最多耗时1小时、或在GPU上耗时数小时完成复现，且均从完全相同的预训练模型出发。⁷我们在第4节对应的子章节中描述了各任务的具体细节。更多细节请参见附录A.5。

4 实验

本节展示 BERT 在 11 项 NLP 任务上的微调结果。

4.1 GLUE

通用语言理解评估 (GLUE) 基准 (Wang 等人, 2018a) 是一组多样化的自然语言理解任务集合。GLUE 数据集的详细说明见附录 B.1。

在 GLUE 上微调时，我们按第3节所述方式表示输入序列（单句或句对），并使用对应于首输入标记 ([CLS]) 的最终隐藏向量 $C \in \mathbb{R}^H$ 作为聚合表征。微调期间引入的唯一新参数为分类层权重 $W \in \mathbb{R}^{K \times H}$ ，其中 K 为标签数量。我们基于 C 和 W 计算标准分类损失，即 $\log(\text{softmax}(CW^T))$ 。

⁷例如，BERT SQuAD 模型可在单块 Cloud TPU 上约 30 分钟内完成训练，达到开发集 F1 分数 91.0%。

⁸参见 <https://gluebenchmark.com/faq> 中第 (10) 条。

System	MNLI-(m/mm) 392k	QQP 363k	QNLI 108k	SST-2 67k	CoLA 8.5k	STS-B 5.7k	MRPC 3.5k	RTE 2.5k	Average
Pre-OpenAI SOTA	80.6/80.1	66.1	82.3	93.2	35.0	81.0	86.0	61.7	74.0
BiLSTM+ELMo+Attn	76.4/76.1	64.8	79.8	90.4	36.0	73.3	84.9	56.8	71.0
OpenAI GPT	82.1/81.4	70.3	87.4	91.3	45.4	80.0	82.3	56.0	75.1
BERT _{BASE}	84.6/83.4	71.2	90.5	93.5	52.1	85.8	88.9	66.4	79.6
BERT _{LARGE}	86.7/85.9	72.1	92.7	94.9	60.5	86.5	89.3	70.1	82.1

Table 1: GLUE Test results, scored by the evaluation server (<https://gluebenchmark.com/leaderboard>). The number below each task denotes the number of training examples. The “Average” column is slightly different than the official GLUE score, since we exclude the problematic WNLI set.⁸ BERT and OpenAI GPT are single-model, single task. F1 scores are reported for QQP and MRPC, Spearman correlations are reported for STS-B, and accuracy scores are reported for the other tasks. We exclude entries that use BERT as one of their components.

We use a batch size of 32 and fine-tune for 3 epochs over the data for all GLUE tasks. For each task, we selected the best fine-tuning learning rate (among $5e-5$, $4e-5$, $3e-5$, and $2e-5$) on the Dev set. Additionally, for BERT_{LARGE} we found that fine-tuning was sometimes unstable on small datasets, so we ran several random restarts and selected the best model on the Dev set. With random restarts, we use the same pre-trained checkpoint but perform different fine-tuning data shuffling and classifier layer initialization.⁹

Results are presented in Table 1. Both BERT_{BASE} and BERT_{LARGE} outperform all systems on all tasks by a substantial margin, obtaining 4.5% and 7.0% respective average accuracy improvement over the prior state of the art. Note that BERT_{BASE} and OpenAI GPT are nearly identical in terms of model architecture apart from the attention masking. For the largest and most widely reported GLUE task, MNLI, BERT obtains a 4.6% absolute accuracy improvement. On the official GLUE leaderboard¹⁰, BERT_{LARGE} obtains a score of 80.5, compared to OpenAI GPT, which obtains 72.8 as of the date of writing.

We find that BERT_{LARGE} significantly outperforms BERT_{BASE} across all tasks, especially those with very little training data. The effect of model size is explored more thoroughly in Section 5.2.

4.2 SQuAD v1.1

The Stanford Question Answering Dataset (SQuAD v1.1) is a collection of 100k crowd-sourced question/answer pairs (Rajpurkar et al., 2016). Given a question and a passage from

Wikipedia containing the answer, the task is to predict the answer text span in the passage.

As shown in Figure 1, in the question answering task, we represent the input question and passage as a single packed sequence, with the question using the A embedding and the passage using the B embedding. We only introduce a start vector $S \in \mathbb{R}^H$ and an end vector $E \in \mathbb{R}^H$ during fine-tuning. The probability of word i being the start of the answer span is computed as a dot product between T_i and S followed by a softmax over all of the words in the paragraph: $P_i = \frac{e^{S \cdot T_i}}{\sum_j e^{S \cdot T_j}}$.

The analogous formula is used for the end of the answer span. The score of a candidate span from position i to position j is defined as $S \cdot T_i + E \cdot T_j$, and the maximum scoring span where $j \geq i$ is used as a prediction. The training objective is the sum of the log-likelihoods of the correct start and end positions. We fine-tune for 3 epochs with a learning rate of $5e-5$ and a batch size of 32.

Table 2 shows top leaderboard entries as well as results from top published systems (Seo et al., 2017; Clark and Gardner, 2018; Peters et al., 2018a; Hu et al., 2018). The top results from the SQuAD leaderboard do not have up-to-date public system descriptions available,¹¹ and are allowed to use any public data when training their systems. We therefore use modest data augmentation in our system by first fine-tuning on TriviaQA (Joshi et al., 2017) before fine-tuning on SQuAD.

Our best performing system outperforms the top leaderboard system by +1.5 F1 in ensembling and +1.3 F1 as a single system. In fact, our single BERT model outperforms the top ensemble system in terms of F1 score. Without TriviaQA fine-

⁹The GLUE data set distribution does not include the Test labels, and we only made a single GLUE evaluation server submission for each of BERT_{BASE} and BERT_{LARGE}.

¹⁰<https://gluebenchmark.com/leaderboard>

¹¹QANet is described in Yu et al. (2018), but the system has improved substantially after publication.

系统	MNLI-(m/mm)	QQP	QNLI	SST-2	CoLA	STS-B	MRPC	RTE	平均	392k	363k	108k	67k
	8.5k	5.7k	3.5k	2.5k	—								
OpenAI 发布前的最优方法													
BiLSTM+ELMo+Attn	76.4/76.1	64.8	79.8	90.4	36.0	73.3	84.9	56.8	71.0				
OpenAI GPT	82.1/81.4	70.3	87.4	91.3	45.4	80.0	82.3	56.0	75.1				
BERT _{BASE}	84.6/83.4	71.2	90.5	93.5	52.1	85.8	88.9	66.4	79.6				
BERT _{LARGE}	86.7/85.9	72.1	92.7	94.9	60.5	86.5	89.3	70.1	82.1				

表1: GLUE 测试集结果 (由评估服务器评分, <https://gluebenchmark.com/leaderboard>)。各任务下方数字表示训练样本数量。‘平均’列与官方 GLUE 分数略有差异, 因为我们排除了存在问题的 WNLI 数据集。⁸ BERT 和 OpenAI GPT 均为单模型、单任务。QQP 和 MRPC 报告 F1 分数, STS-B 报告 Spearman 相关系数, 其余任务报告准确率。我们排除了将 BERT 作为其组件之一的参赛条目。

所有 GLUE 任务均采用批大小 32 并微调 3 轮。对每项任务, 我们在开发集上从 $5e-5$ 、 $4e-5$ 、 $3e-5$ 和 $2e-5$ 中选出最优微调学习率。此外, 我们发现 BERT_{LARGE} 在小型数据集上微调有时不稳定, 因此对每个任务进行了多次随机重启, 并在开发集上选取最优模型。随机重启时, 我们使用相同的预训练检查点, 但对微调数据打乱顺序及分类器层初始化采用不同随机种子。⁹

结果见表1。BERT_{BASE}和 BERT_{LARGE} 在所有任务上均大幅超越此前所有系统, 分别较先前最优方法平均准确率提升 4.5% 和 7.0%。注意, BERT_{BASE} 与 OpenAI GPT 在模型结构上几乎完全相同, 区别仅在于注意力掩码机制。在规模最大、报道最广泛的 GLUE 任务 MNLI 上, BERT 实现了 4.6% 的绝对准确率提升。在官方 GLUE 排行榜¹⁰上, BERT_{LARGE} 得分为 80.5, 而截至本文撰写时, OpenAI GPT 得分为 72.8。

我们发现, BERT_{LARGE} 在所有任务上均显著优于 BERT_{BASE}, 尤其在训练数据极少的任务上表现更为突出。模型规模的影响将在第 5.2 节中更深入探讨。

4.2 SQuAD v1.1

斯坦福问答数据集 (SQuAD v1.1) 包含 10 万组众包问答对 (Rajpurkar 等人, 2016)。给定一个问题及一段来自

包含答案的维基百科段落, 任务是预测该段落中答案所对应的文本片段。

如图1所示, 在问答任务中, 我们将输入问题与段落表示为单一打包序列, 问题部分使用 A 嵌入, 段落部分使用 B 嵌入。微调期间仅引入一个起始向量 $S \in \mathbb{R}^H$ 和一个结束向量 $E \in \mathbb{R}^H$ 。词 i 作为答案片段起始位置的概率通过 T_i 与 S 的点积并经段落内所有词的 softmax 归一化得到: $P_i = e^{S \cdot T_i}$

答案片段结尾位置采用类似公式计算。候选片段 (从位置 i 到位置 j) 的得分定义为 $S \cdot T_i + E \cdot T_j$, 其中 $j \geq i$ 的最高分片段被选作预测结果。训练目标是正确起始位置与结束位置对数似然之和。我们以学习率 $5e-5$ 、批大小 32 微调 3 轮。

表2列出了排行榜前列条目以及顶尖已发表系统的结果 (Seo 等人, 2017; Clark 和 Gardner, 2018; Peters 等人, 2018a; Hu 等人, 2018)。SQuAD 排行榜上的最优结果目前尚无最新公开的系统描述,¹¹ 且允许在训练过程中使用任意公开数据。因此, 我们在系统中采用了适度的数据增强策略: 先在 TriviaQA (Joshi 等人, 2017) 上进行微调, 再于 SQuAD 上进一步微调。

我们性能最佳的系统, 在集成模式下 F1 分数高出排行榜榜首系统 +1.5, 在单模型模式下高出 +1.3。事实上, 我们的单模型 BERT 在 F1 分数上已超越排行榜榜首的集成系统。未使用 TriviaQA 进行微调——

⁹ GLUE 数据集发布版不包含测试集标签, 我们仅为 BERT_{BASE} 和 BERT_{LARGE} 各提交了一次 GLUE 评估服务器结果。

¹⁰ <https://gluebenchmark.com/leaderboard>

¹¹ QANet 在 Yu 等人 (2018) 中有所描述, 但该系统在发表后取得了显著改进。

System	Dev		Test	
	EM	F1	EM	F1
Top Leaderboard Systems (Dec 10th, 2018)				
Human	-	-	82.3	91.2
#1 Ensemble - nlnet	-	-	86.0	91.7
#2 Ensemble - QANet	-	-	84.5	90.5
Published				
BiDAF+ELMo (Single)	-	85.6	-	85.8
R.M. Reader (Ensemble)	81.2	87.9	82.3	88.5
Ours				
BERT _{BASE} (Single)	80.8	88.5	-	-
BERT _{LARGE} (Single)	84.1	90.9	-	-
BERT _{LARGE} (Ensemble)	85.8	91.8	-	-
BERT _{LARGE} (Sgl.+TriviaQA)	84.2	91.1	85.1	91.8
BERT _{LARGE} (Ens.+TriviaQA)	86.2	92.2	87.4	93.2

Table 2: SQuAD 1.1 results. The BERT ensemble is 7x systems which use different pre-training checkpoints and fine-tuning seeds.

System	Dev		Test	
	EM	F1	EM	F1
Top Leaderboard Systems (Dec 10th, 2018)				
Human	86.3	89.0	86.9	89.5
#1 Single - MIR-MRC (F-Net)	-	-	74.8	78.0
#2 Single - nlnet	-	-	74.2	77.1
Published				
unet (Ensemble)	-	-	71.4	74.9
SLQA+ (Single)	-	-	71.4	74.4
Ours				
BERT _{LARGE} (Single)	78.7	81.9	80.0	83.1

Table 3: SQuAD 2.0 results. We exclude entries that use BERT as one of their components.

tuning data, we only lose 0.1-0.4 F1, still outperforming all existing systems by a wide margin.¹²

4.3 SQuAD v2.0

The SQuAD 2.0 task extends the SQuAD 1.1 problem definition by allowing for the possibility that no short answer exists in the provided paragraph, making the problem more realistic.

We use a simple approach to extend the SQuAD v1.1 BERT model for this task. We treat questions that do not have an answer as having an answer span with start and end at the [CLS] token. The probability space for the start and end answer span positions is extended to include the position of the [CLS] token. For prediction, we compare the score of the no-answer span: $s_{\text{null}} = S \cdot C + E \cdot C$ to the score of the best non-null span

¹²The TriviaQA data we used consists of paragraphs from TriviaQA-Wiki formed of the first 400 tokens in documents, that contain at least one of the provided possible answers.

System	Dev	Test
ESIM+GloVe	51.9	52.7
ESIM+ELMo	59.1	59.2
OpenAI GPT	-	78.0
BERT _{BASE}	81.6	-
BERT _{LARGE}	86.6	86.3
Human (expert) [†]	-	85.0
Human (5 annotations) [†]	-	88.0

Table 4: SWAG Dev and Test accuracies. [†]Human performance is measured with 100 samples, as reported in the SWAG paper.

$s_{i,j} = \max_{j \geq i} S \cdot T_i + E \cdot T_j$. We predict a non-null answer when $s_{i,j} > s_{\text{null}} + \tau$, where the threshold τ is selected on the dev set to maximize F1. We did not use TriviaQA data for this model. We fine-tuned for 2 epochs with a learning rate of 5e-5 and a batch size of 48.

The results compared to prior leaderboard entries and top published work (Sun et al., 2018; Wang et al., 2018b) are shown in Table 3, excluding systems that use BERT as one of their components. We observe a +5.1 F1 improvement over the previous best system.

4.4 SWAG

The Situations With Adversarial Generations (SWAG) dataset contains 113k sentence-pair completion examples that evaluate grounded common-sense inference (Zellers et al., 2018). Given a sentence, the task is to choose the most plausible continuation among four choices.

When fine-tuning on the SWAG dataset, we construct four input sequences, each containing the concatenation of the given sentence (sentence A) and a possible continuation (sentence B). The only task-specific parameters introduced is a vector whose dot product with the [CLS] token representation C denotes a score for each choice which is normalized with a softmax layer.

We fine-tune the model for 3 epochs with a learning rate of 2e-5 and a batch size of 16. Results are presented in Table 4. BERT_{LARGE} outperforms the authors’ baseline ESIM+ELMo system by +27.1% and OpenAI GPT by 8.3%.

5 Ablation Studies

In this section, we perform ablation experiments over a number of facets of BERT in order to better understand their relative importance. Additional

系统	开发集		测试集	
	EM	F1	EM	F1
排行榜领先系统 (2018年12月10日) 人类水平				
#1 集成 — nlnet				
#2 集成 — QANet				
已发表工作				
BiDAF+ELMo (单模型)				
R.M. Reader (集成)	81.2	87.9	82.3	88.5
我们的方法				
BERTBASE (单模型)				
BERTLARGE (单模型)				
BERTLARGE (集成)	85.8	91.8	—	—
BERTLARGE (单模型 + TriviaQA)	84.2	91.1	85.1	91.8
BERTLARGE (集成 + TriviaQA)	86.2	92.2	87.4	93.2

表2: SQuAD 1.1 结果。BERT 集成模型由 7 个系统组成, 这些系统使用不同的预训练检查点和微调随机种子。

系统	开发集		测试集	
	EM	F1	EM	F1
排行榜领先系统 (2018年12月10日) 人类水平				
#1 单模型 — MIR-MRC (F-Net)	—	—	74.8	78.0
#2 单模型 — nlnet	—	—	74.2	77.1
已发表				
unet (集成)				
SLQA+ (单模型)	—	—	71.4	74.4
我们的方法				
BERTLARGE (单模型)				

表3: SQuAD 2.0结果。我们排除了将BERT作为组件之一的系统。

微调数据, 我们仅损失0.1–0.4 F1值, 但仍大幅超越所有现有系统。¹²

4.3 SQuAD v2.0

SQuAD 2.0任务在SQuAD 1.1问题定义基础上加以扩展, 允许所提供段落中不存在简短答案, 从而使问题设定更贴近现实。

我们采用一种简单方法将SQuAD v1.1的BERT模型扩展至此任务。我们将无答案的问题视为答案跨度起止位置均在[CLS]词元处。答案起始与终止位置的概率空间被扩展, 以包含[CLS]词元的位置。在预测时, 我们将无答案跨度的得分 $s_{\text{null}} = S \cdot C + E \cdot C$ 与最优非空跨度得分进行比较。

¹²我们所用的TriviaQA数据来自TriviaQA-Wiki段落, 取自文档中前400个词元, 且至少包含一个所提供可能答案。

系统	开发集	测试集
ESIM+GloVe	51.9	52.7
ESIM+ELMo	59.1	59.2
OpenAI GPT	-	78.0
BERT _{BASE}	81.6	-
BERT _{LARGE}	86.6	86.3
人类 (专家) [†]	-	85.0
人类 (5次标注) [†]	-	88.0

表4: SWAG开发集与测试集准确率。[†]人类性能基于SWAG论文中报告的100个样本测量。

$\hat{s}_{ij} = \max_j \{iS \cdot T_i + E \cdot T_j\}$ 。当 $\hat{s}_{ij} > s_{\text{null}} + \tau$ 时, 我们预测存在非空答案, 其中阈值 τ 在开发集上选定, 以最大化F1值。本模型未使用TriviaQA数据。我们以 $5e-5$ 的学习率和48的批处理大小进行了2轮微调。

与先排行榜条目及顶级已发表工作 (Sun等, 2018; Wang等, 2018b) 的对比结果见表3 (排除将BERT作为组件之一的系统)。我们观察到相较此前最佳系统, F1值提升了+5.1。

4.4 SWAG

Situations With Adversarial Generations (SWAG) 数据集包含11.3万个句子对补全样例, 用于评估具身常识推理 (Zellers等, 2018)。给定一个句子, 任务是从四个选项中选出最合理的续句。

在SWAG数据集上微调时, 我们构建四个输入序列, 每个序列均包含给定句子 (句子A) 与一个可能续句 (句子B) 的拼接。唯一引入的任务特定参数是一个向量, 其与[CLS]词元表征C的点积为每个选项生成一个得分, 并经softmax层归一化。

我们以 $2e-5$ 的学习率和16的批处理大小对模型进行3轮微调。结果见表4。BERTLARGE相较于作者基线 ESIM+ELMo 系统提升+27.1%, 较OpenAI GPT提升8.3%。

5 消融研究

在本节中, 我们针对BERT的多个方面开展消融实验, 以更深入理解各要素的相对重要性。更多实验细节见附录C。

Tasks	MNLI-m (Acc)	Dev Set			
		QNLI (Acc)	MRPC (Acc)	SST-2 (Acc)	SQuAD (F1)
BERT _{BASE}	84.4	88.4	86.7	92.7	88.5
No NSP	83.9	84.9	86.5	92.6	87.9
LTR & No NSP	82.1	84.3	77.5	92.1	77.8
+ BiLSTM	82.1	84.1	75.7	91.6	84.9

Table 5: Ablation over the pre-training tasks using the BERT_{BASE} architecture. “No NSP” is trained without the next sentence prediction task. “LTR & No NSP” is trained as a left-to-right LM without the next sentence prediction, like OpenAI GPT. “+ BiLSTM” adds a randomly initialized BiLSTM on top of the “LTR + No NSP” model during fine-tuning.

ablation studies can be found in Appendix C.

5.1 Effect of Pre-training Tasks

We demonstrate the importance of the deep bidirectionality of BERT by evaluating two pre-training objectives using exactly the same pre-training data, fine-tuning scheme, and hyperparameters as BERT_{BASE}:

No NSP: A bidirectional model which is trained using the “masked LM” (MLM) but without the “next sentence prediction” (NSP) task.

LTR & No NSP: A left-context-only model which is trained using a standard Left-to-Right (LTR) LM, rather than an MLM. The left-only constraint was also applied at fine-tuning, because removing it introduced a pre-train/fine-tune mismatch that degraded downstream performance. Additionally, this model was pre-trained without the NSP task. This is directly comparable to OpenAI GPT, but using our larger training dataset, our input representation, and our fine-tuning scheme.

We first examine the impact brought by the NSP task. In Table 5, we show that removing NSP hurts performance significantly on QNLI, MNLI, and SQuAD 1.1. Next, we evaluate the impact of training bidirectional representations by comparing “No NSP” to “LTR & No NSP”. The LTR model performs worse than the MLM model on all tasks, with large drops on MRPC and SQuAD.

For SQuAD it is intuitively clear that a LTR model will perform poorly at token predictions, since the token-level hidden states have no right-side context. In order to make a good faith attempt at strengthening the LTR system, we added a randomly initialized BiLSTM on top. This does significantly improve results on SQuAD, but the

results are still far worse than those of the pre-trained bidirectional models. The BiLSTM hurts performance on the GLUE tasks.

We recognize that it would also be possible to train separate LTR and RTL models and represent each token as the concatenation of the two models, as ELMo does. However: (a) this is twice as expensive as a single bidirectional model; (b) this is non-intuitive for tasks like QA, since the RTL model would not be able to condition the answer on the question; (c) this is strictly less powerful than a deep bidirectional model, since it can use both left and right context at every layer.

5.2 Effect of Model Size

In this section, we explore the effect of model size on fine-tuning task accuracy. We trained a number of BERT models with a differing number of layers, hidden units, and attention heads, while otherwise using the same hyperparameters and training procedure as described previously.

Results on selected GLUE tasks are shown in Table 6. In this table, we report the average Dev Set accuracy from 5 random restarts of fine-tuning. We can see that larger models lead to a strict accuracy improvement across all four datasets, even for MRPC which only has 3,600 labeled training examples, and is substantially different from the pre-training tasks. It is also perhaps surprising that we are able to achieve such significant improvements on top of models which are already quite large relative to the existing literature. For example, the largest Transformer explored in Vaswani et al. (2017) is (L=6, H=1024, A=16) with 100M parameters for the encoder, and the largest Transformer we have found in the literature is (L=64, H=512, A=2) with 235M parameters (Al-Rfou et al., 2018). By contrast, BERT_{BASE} contains 110M parameters and BERT_{LARGE} contains 340M parameters.

It has long been known that increasing the model size will lead to continual improvements on large-scale tasks such as machine translation and language modeling, which is demonstrated by the LM perplexity of held-out training data shown in Table 6. However, we believe that this is the first work to demonstrate convincingly that scaling to extreme model sizes also leads to large improvements on very small scale tasks, provided that the model has been sufficiently pre-trained. Peters et al. (2018b) presented

任务	开发集				
	MNLI-m (准确率)	QNLI (准确率)	MRPC (准确率)	SST-2 (准确率)	SQuAD (F1)
BERT _{BASE}	84.4	88.4	86.7	92.7	88.5
No NSP	83.9	84.9	86.5	92.6	87.9
LTR & No NSP	82.1	84.3	77.5	92.1	77.8
+ BiLSTM	82.1	84.1	75.7	91.6	84.9

表5：基于BERTBASE架构的预训练任务消融实验。No NSP指未使用下一句预测任务训练的模型。LTR & No NSP指类似OpenAI GPT的左到右语言模型，未使用下一句预测任务。+ BiLSTM指在LTR + No NSP模型微调阶段，于其顶部添加一个随机初始化的BiLSTM。

消融实验详见附录C。

5.1 预训练任务的影响

我们通过采用与BERTBASE完全相同的预训练数据、微调方案及超参数来评估两种预训练目标，以此验证BERT深度双向性的重要性：

No NSP：一种双向模型，其采用掩码语言模型（MLM）进行训练，但未包含下一句预测（NSP）任务。

LTR & No NSP：一种仅使用左侧上下文的模型，其采用标准的左到右（LTR）语言模型（LM）而非掩码语言模型（MLM）进行训练。在微调阶段也施加了仅左侧约束，因为取消该约束会引发预训练/微调不匹配问题，从而降低下游任务性能。此外，该模型在预训练阶段未使用NSP任务。该模型可直接与OpenAI GPT对比，但采用了我们更大的训练数据集、我们的输入表征方式以及我们的微调方案。

我们首先考察下一句预测（NSP）任务带来的影响。表5显示，在QNLI、MNLI和SQuAD 1.1任务上移除NSP会显著损害性能。接着，我们通过对比“**No NSP**”与“**LTR & No NSP**”来评估训练双向表征的影响。LTR模型在所有任务上的表现均劣于MLM模型，其中MRPC和SQuAD任务的性能下降尤为明显。

对于SQuAD任务，LTR模型在词元级预测中表现欠佳是显而易见的，因为词元级隐状态缺乏右侧上下文。为尽可能增强LTR系统，我们在其顶部添加了一个随机初始化的BiLSTM。这确实显著提升了SQuAD上的结果，但其性能仍远低于预训练双向模型。

其结果仍远逊于预训练双向模型。BiLSTM在GLUE任务上损害了性能。

我们意识到，也可分别训练独立的左到右（LTR）和右到左（RTL）模型，并像ELMo那样将每个词元表示为两个模型输出的拼接。然而：（a）该方法的计算开销是单个双向模型的两倍；（b）对问答（QA）等任务而言不够直观，因为RTL模型无法依据问题来约束答案生成；（c）该方法严格弱于深度双向模型，因为后者能在每一层同时利用左右两侧上下文。

5.2 模型规模的影响

在本节中，我们探究模型规模对微调任务准确率的影响。我们训练了若干BERT模型，其层数、隐藏单元数及注意力头数各不相同，其余超参数与训练流程则与前述保持一致。

部分GLUE任务的结果如表6所示。在该表中，我们报告了5次随机初始化微调的开发集准确率平均值。可以看出，更大的模型在全部四个数据集上均带来严格的准确率提升，即便对于仅有3600个标注训练样本、且与预训练任务差异显著的MRPC任务亦是如此。另一个或许令人惊讶之处在于，我们在相较现有文献而言已相当庞大的模型基础上，仍实现了如此显著的性能提升。例如，Vaswani等（2017）探索的最大Transformer模型为（ $L=6, H=1024, A=16$ ），其编码器含1亿参数；而据我们所见，文献中最大的Transformer模型为（ $L=64, H=512, A=2$ ），含2.35亿参数（Al-Rfou等，2018）。相比之下，BERTBASE含1.1亿参数，BERTLARGE含3.4亿参数。

长期以来人们已知，增大模型规模将持续提升机器翻译和语言建模等大规模任务的性能，这一点可由表6中所示的预留训练数据的语言模型困惑度（LM perplexity）得到印证。然而，我们认为这是首项有力证实：只要模型经过充分预训练，将模型规模扩展至极端程度，同样可在极小规模任务上带来显著性能提升。Peters等（2018b）曾提出

mixed results on the downstream task impact of increasing the pre-trained bi-LM size from two to four layers and Melamud et al. (2016) mentioned in passing that increasing hidden dimension size from 200 to 600 helped, but increasing further to 1,000 did not bring further improvements. Both of these prior works used a feature-based approach — we hypothesize that when the model is fine-tuned directly on the downstream tasks and uses only a very small number of randomly initialized additional parameters, the task-specific models can benefit from the larger, more expressive pre-trained representations even when downstream task data is very small.

5.3 Feature-based Approach with BERT

All of the BERT results presented so far have used the fine-tuning approach, where a simple classification layer is added to the pre-trained model, and all parameters are jointly fine-tuned on a downstream task. However, the feature-based approach, where fixed features are extracted from the pre-trained model, has certain advantages. First, not all tasks can be easily represented by a Transformer encoder architecture, and therefore require a task-specific model architecture to be added. Second, there are major computational benefits to pre-compute an expensive representation of the training data once and then run many experiments with cheaper models on top of this representation.

In this section, we compare the two approaches by applying BERT to the CoNLL-2003 Named Entity Recognition (NER) task (Tjong Kim Sang and De Meulder, 2003). In the input to BERT, we use a case-preserving WordPiece model, and we include the maximal document context provided by the data. Following standard practice, we formulate this as a tagging task but do not use a CRF

Hyperparams				Dev Set Accuracy		
#L	#H	#A	LM (ppl)	MNLI-m	MRPC	SST-2
3	768	12	5.84	77.9	79.8	88.4
6	768	3	5.24	80.6	82.2	90.7
6	768	12	4.68	81.9	84.8	91.3
12	768	12	3.99	84.4	86.7	92.9
12	1024	16	3.54	85.7	86.9	93.3
24	1024	16	3.23	86.6	87.8	93.7

Table 6: Ablation over BERT model size. #L = the number of layers; #H = hidden size; #A = number of attention heads. “LM (ppl)” is the masked LM perplexity of held-out training data.

System	Dev F1	Test F1
ELMo (Peters et al., 2018a)	95.7	92.2
CVT (Clark et al., 2018)	-	92.6
CSE (Akbik et al., 2018)	-	93.1
Fine-tuning approach		
BERT _{LARGE}	96.6	92.8
BERT _{BASE}	96.4	92.4
Feature-based approach (BERT _{BASE})		
Embeddings	91.0	-
Second-to-Last Hidden	95.6	-
Last Hidden	94.9	-
Weighted Sum Last Four Hidden	95.9	-
Concat Last Four Hidden	96.1	-
Weighted Sum All 12 Layers	95.5	-

Table 7: CoNLL-2003 Named Entity Recognition results. Hyperparameters were selected using the Dev set. The reported Dev and Test scores are averaged over 5 random restarts using those hyperparameters.

layer in the output. We use the representation of the first sub-token as the input to the token-level classifier over the NER label set.

To ablate the fine-tuning approach, we apply the feature-based approach by extracting the activations from one or more layers *without* fine-tuning any parameters of BERT. These contextual embeddings are used as input to a randomly initialized two-layer 768-dimensional BiLSTM before the classification layer.

Results are presented in Table 7. BERT_{LARGE} performs competitively with state-of-the-art methods. The best performing method concatenates the token representations from the top four hidden layers of the pre-trained Transformer, which is only 0.3 F1 behind fine-tuning the entire model. This demonstrates that BERT is effective for both fine-tuning and feature-based approaches.

6 Conclusion

Recent empirical improvements due to transfer learning with language models have demonstrated that rich, unsupervised pre-training is an integral part of many language understanding systems. In particular, these results enable even low-resource tasks to benefit from deep unidirectional architectures. Our major contribution is further generalizing these findings to deep *bidirectional* architectures, allowing the same pre-trained model to successfully tackle a broad set of NLP tasks.

先前研究在下游任务上的效果呈现混合结果：Melamud等（2016）曾简要提及，将预训练双向语言模型（bi-LM）的层数从两层增至四层，以及将隐藏层维度从200提升至600有助于性能提升，但进一步增至1000则未带来进一步改进。这两项前期工作均采用基于特征的方法——我们假设，当模型直接在下游任务上进行微调、且仅引入极少量随机初始化的额外参数时，即使下游任务的数据量非常小，任务特定模型仍可受益于更大、更具表达能力的预训练表征。

5.3 基于特征的方法与BERT

截至目前所展示的所有BERT结果均采用了微调方法，即在预训练模型上添加一个简单的分类层，并在下游任务上联合微调所有参数。然而，基于特征的方法（即从预训练模型中提取固定特征）具有某些优势：第一，并非所有任务都能轻易地用Transformer编码器架构表示，因此需要额外添加特定于任务的模型架构；第二，预先计算一次训练数据的昂贵表征，然后在此表征之上运行多个计算成本较低的实验，具有显著的计算优势。

在本节中，我们通过将BERT应用于CoNLL-2003命名实体识别（NER）任务（Tjong Kim Sang和De Meulder, 2003）来比较这两种方法。在BERT的输入中，我们采用区分大小写的WordPiece模型，并纳入数据所提供的最大文档上下文。遵循标准做法，我们将该任务建模为序列标注任务，但不使用CRF

超参数	开发集准确率									
#L #H #A LM (ppl)	MNLI-m	MRPC	SST-2	3						
768 12 5.84 77.9	79.8	88.4	6	768	3	5.24	80.6			
82.2 90.7 6	768	12	4.68	81.9	84.8	91.3	12			
768 12 3.99	84.4	86.7	92.9	12	1024	16	3.54			
85.7 86.9 93.3	24	1024	16	3.23	86.6	87.8				
93.7										

表6：BERT模型尺寸的消融实验。#L = 层数；#H = 隐藏层维度；#A = 注意力头数量。LM (ppl) 为保留训练数据的掩码语言模型困惑度。

系统	开发集F1值 测试集F1值	
ELMo (Peters等, 2018a)		
CVT (Clark等, 2018)		
CSE (Akbik等, 2018)		93.1
微调方法		
BERT _{LARGE}	96.6	92.8
BERT _{BASE}	96.4	92.4
基于特征的方法 (BERT _{BASE})		
嵌入		
倒数第二层隐藏层		
最后一层隐藏层		
加权求和最后四层隐藏层	95.9	—— 拼接最
最后四层隐藏层	96.1	—— 加权求和全部12层

表7：CoNLL-2003命名实体识别结果。超参数通过开发集选定。报告的开发集与测试集得分取自5次随机重启的平均值。使用这些超参数进行5次随机重启。

输出层中的某一层。我们使用第一个子词元的表征作为面向NER标签集的词元级分类器的输入。

为消融微调方法，我们采用基于特征的方法：提取一个或多个层的激活值，且不微调BERT的任何参数。这些上下文嵌入被用作随机初始化的两层768维双向LSTM的输入，再接入分类层。

结果见表7。BERT_{LARGE}在性能上与当前最优方法具有竞争力。表现最佳的方法将预训练Transformer顶层四个隐藏层的词元表征进行拼接，其性能仅比微调整个模型低0.3 F1值。这表明BERT既适用于微调，也适用于基于特征的方法。

6 结论

近期得益于语言模型迁移学习所取得的经验性改进表明，丰富的无监督预训练是诸多语言理解系统不可或缺的一部分。特别是，这些成果使得即使是低资源任务也能从深度单向架构中获益。我们的主要贡献在于进一步将这些发现推广至深度双向架构，从而使同一预训练模型能够成功应对一系列广泛的自然语言处理任务。

References

- Alan Akbik, Duncan Blythe, and Roland Vollgraf. 2018. Contextual string embeddings for sequence labeling. In *Proceedings of the 27th International Conference on Computational Linguistics*, pages 1638–1649.
- Rami Al-Rfou, Dokook Choe, Noah Constant, Mandy Guo, and Llion Jones. 2018. Character-level language modeling with deeper self-attention. *arXiv preprint arXiv:1808.04444*.
- Rie Kubota Ando and Tong Zhang. 2005. A framework for learning predictive structures from multiple tasks and unlabeled data. *Journal of Machine Learning Research*, 6(Nov):1817–1853.
- Luisa Bentivogli, Bernardo Magnini, Ido Dagan, Hoa Trang Dang, and Danilo Giampiccolo. 2009. The fifth PASCAL recognizing textual entailment challenge. In *TAC. NIST*.
- John Blitzer, Ryan McDonald, and Fernando Pereira. 2006. Domain adaptation with structural correspondence learning. In *Proceedings of the 2006 conference on empirical methods in natural language processing*, pages 120–128. Association for Computational Linguistics.
- Samuel R. Bowman, Gabor Angeli, Christopher Potts, and Christopher D. Manning. 2015. A large annotated corpus for learning natural language inference. In *EMNLP*. Association for Computational Linguistics.
- Peter F Brown, Peter V Desouza, Robert L Mercer, Vincent J Della Pietra, and Jenifer C Lai. 1992. Class-based n-gram models of natural language. *Computational linguistics*, 18(4):467–479.
- Daniel Cer, Mona Diab, Eneko Agirre, Inigo Lopez-Gazpio, and Lucia Specia. 2017. [Semeval-2017 task 1: Semantic textual similarity multilingual and crosslingual focused evaluation](#). In *Proceedings of the 11th International Workshop on Semantic Evaluation (SemEval-2017)*, pages 1–14, Vancouver, Canada. Association for Computational Linguistics.
- Ciprian Chelba, Tomas Mikolov, Mike Schuster, Qi Ge, Thorsten Brants, Philipp Koehn, and Tony Robinson. 2013. One billion word benchmark for measuring progress in statistical language modeling. *arXiv preprint arXiv:1312.3005*.
- Z. Chen, H. Zhang, X. Zhang, and L. Zhao. 2018. [Quora question pairs](#).
- Christopher Clark and Matt Gardner. 2018. Simple and effective multi-paragraph reading comprehension. In *ACL*.
- Kevin Clark, Minh-Thang Luong, Christopher D Manning, and Quoc Le. 2018. Semi-supervised sequence modeling with cross-view training. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 1914–1925.
- Ronan Collobert and Jason Weston. 2008. A unified architecture for natural language processing: Deep neural networks with multitask learning. In *Proceedings of the 25th international conference on Machine learning*, pages 160–167. ACM.
- Alexis Conneau, Douwe Kiela, Holger Schwenk, Loïc Barrault, and Antoine Bordes. 2017. [Supervised learning of universal sentence representations from natural language inference data](#). In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, pages 670–680, Copenhagen, Denmark. Association for Computational Linguistics.
- Andrew M Dai and Quoc V Le. 2015. Semi-supervised sequence learning. In *Advances in neural information processing systems*, pages 3079–3087.
- J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei. 2009. ImageNet: A Large-Scale Hierarchical Image Database. In *CVPR09*.
- William B Dolan and Chris Brockett. 2005. Automatically constructing a corpus of sentential paraphrases. In *Proceedings of the Third International Workshop on Paraphrasing (IWP2005)*.
- William Fedus, Ian Goodfellow, and Andrew M Dai. 2018. Maskgan: Better text generation via filling in the... *arXiv preprint arXiv:1801.07736*.
- Dan Hendrycks and Kevin Gimpel. 2016. [Bridging nonlinearities and stochastic regularizers with gaussian error linear units](#). *CoRR*, abs/1606.08415.
- Felix Hill, Kyunghyun Cho, and Anna Korhonen. 2016. Learning distributed representations of sentences from unlabelled data. In *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics.
- Jeremy Howard and Sebastian Ruder. 2018. [Universal language model fine-tuning for text classification](#). In *ACL*. Association for Computational Linguistics.
- Minghao Hu, Yuxing Peng, Zhen Huang, Xipeng Qiu, Furu Wei, and Ming Zhou. 2018. Reinforced mnemonic reader for machine reading comprehension. In *IJCAI*.
- Yacine Jernite, Samuel R. Bowman, and David Sonntag. 2017. [Discourse-based objectives for fast unsupervised sentence representation learning](#). *CoRR*, abs/1705.00557.

参考文献

阿兰·阿克比克 (Alan Akbik)、邓肯·布莱斯 (Duncan Blythe) 和罗兰·沃尔格拉夫 (Roland Vollgraf)。2018。用于序列标注的上下文字符串嵌入。载于《第二十七届国际计算语言学大会论文集》，第1638–1649页。

拉米·阿尔-鲁富 (Rami Al-Rfou)、道库克·崔 (Dokook Choe)、诺亚·康斯坦特 (Noah Constant)、曼迪·郭 (Mandy Guo) 和里昂·琼斯 (Llion Jones)。2018。利用更深层自注意力机制的字符级语言建模。arXiv预印本 arXiv:1808.04444。

理惠·库博塔·安多 (Rie Kubota Ando) 和张彤 (Tong Zhang)。2005。一种从多项任务和未标注数据中学习预测性结构的框架。《机器学习研究杂志》，6(Nov):1817–1853。

路易莎·本蒂沃利 (Luisa Bentivogli)、贝尔纳多·马尼尼 (Bernardo Magnini)、伊多·达甘 (Ido Dagan)、何阿·特朗·邓 (Hoa Trang Dang) 和达尼洛·贾姆皮科洛 (Danilo Giampiccolo)。2009。第五届PASCAL文本蕴含识别挑战赛。载于 TAC。NIST。

约翰·布利策 (John Blitzer)、瑞安·麦克唐纳 (Ryan McDonald) 和费尔南多·佩雷拉 (Fernando Pereira)。2006。利用结构对应学习进行领域自适应。载于《2006年会议论文集》。经验方法自然语言处理会议。自然语言处理，第120–128页。计算语言学协会。

塞缪尔·R·鲍曼 (Samuel R. Bowman)、加博尔·安吉利 (Gabor Angeli)、克里斯托弗·波特 (Christopher Potts) 和克里斯托弗·D·曼宁 (Christopher D. Manning)。2015。用于学习自然语言推理的大规模标注语料库。载于 EMNLP。计算语言学协会。

彼得·F·布朗 (Peter F. Brown)、彼得·V·德索萨 (Peter V. Desouza)、罗伯特·L·默瑟 (Robert L. Mercer)、文森特·J·德拉皮特拉 (Vincent J. Della Pietra) 和珍妮弗·C·莱 (Jennifer C. Lai)。1992。基于类别的n元语法自然语言模型。《计算语言学》，18(4):467–479。

丹尼尔·塞尔 (Daniel Cer)、莫娜·迪亚布 (Mona Diab)、埃内科·阿吉雷 (Eneko Agirre)、伊尼戈·洛佩斯-加西波 (Inigo Lopez-Gazpio) 和露西亚·斯佩恰 (Lucia Specia)。2017。SemEval-2017任务1：语义文本相似度多语言及跨语言聚焦评测。载于《……》。第11届语义国际研讨会。语义评测 (SemEval-2017)，第1–14页，加拿大温哥华。计算语言学协会。

奇普里安·切尔巴 (Ciprian Chelba)、托马斯·米科洛夫 (Tomas Mikolov)、迈克·舒斯特 (Mike Schuster)、齐·葛 (Qi Ge)、托尔斯滕·布兰特 (Thorsten Brants)、菲利普·科恩 (Phillipp Koehn) 和托尼·罗宾逊 (Tony Robinson)。2013。用于衡量统计语言建模进展的十亿词基准测试。arXiv预印本 arXiv:1312.3005。

Z. 陈 (Z. Chen)、H. 张 (H. Zhang)、X. 张 (X. Zhang) 和 L. 赵 (L. Zhao)。2018。Quora问题对。

克里斯托弗·克拉克 (Christopher Clark) 和马特·加德纳 (Matt Gardner)。2018。简单而有效的多段落阅读理解。载于 ACL。

凯文·克拉克 (Kevin Clark)、明·唐·卢翁 (Minh-Thang Luong)、克里斯托弗·D·曼宁 (Christopher D. Manning) 和邱锡南 (Quoc Le)。2018。利用跨视角训练进行半监督序列建模。载于《……》。2018年经验方法自然语言处理会议论文集。自然语言处理方法，第1914–1925页。

罗南·科洛贝尔 (Ronan Collobert) 和杰森·韦斯顿 (Jason Weston)。2008。自然语言处理的统一架构：带多任务学习的深度神经网络。载于《……》。第25届国际会议论文集。机器学习，第160–167页。ACM。

亚历克西斯·孔诺 (Alexis Conneau)、杜威·基拉 (Douwe Kiela)、霍尔格·施文克 (Holger Schwenk)、洛伊克·巴拉尔 (Loïc Barrault) 和安托万·博尔德斯 (Antoine Bordes)。2017。基于自然语言推理数据的通用句子表征监督学习。载于《……》。2017年经验方法自然语言处理会议。自然语言处理，第670–680页，丹麦哥本哈根。计算语言学协会。

安德鲁·M·戴 (Andrew M. Dai) 和邱锡南 (Quoc V. Le)。2015。半监督序列学习。载于《神经信息处理系统进展》，第3079–3087页。

J. 邓 (J. Deng)、W. 东 (W. Dong)、R. 索彻 (R. Socher)、L.-J. 李 (L.-J. Li)、K. 李 (K. Li) 和 L. 费·费 (L. Fei-Fei)。2009。ImageNet：一个大规模分层图像数据库。载于 CVPR09。

威廉·B·多兰 (William B. Dolan) 和克里斯·布罗凯特 (Chris Brockett)。2005。自动构建句子级释义语料库。载于《第三届释义国际研讨会论文集》(IWP2005)。

威廉·费杜斯、伊恩·古德费洛和安德鲁·M·戴。2018。MaskGAN：借助填充空白提升文本生成效果。arXiv预印本 arXiv:1801.07736

丹·亨德里克斯和凯文·吉姆佩尔。2016。用高斯误差线性单元连接非线性与随机正则化。CoRR, abs/1606.08415。费利克斯·希尔、丘成熙和安娜·科罗宁。2016。

从无标签数据中学习句子的分布式表示。载于《2016年会议论文集》。计算语言学协会北美分会年会：人类语言技术。语言技术。计算语言学协会。

杰里米·霍华德和塞巴斯蒂安·鲁德尔。2018。面向文本分类的通用语言模型微调。载于《计算语言学协会年会》(ACL)。计算语言学协会。胡明浩、彭宇星、黄震、邱锡鹏。

付儒、魏和周明。2018。用于机器阅读理解的强化记忆式阅读器。载于《国际人工智能联合会议》(IJCAI)

亚辛·杰尔尼特、塞缪尔·R·鲍曼和大卫·松塔格。2017。基于篇章的目标函数用于快速无监督句子表示学习。CoRR, abs/1705.00557。

- Mandar Joshi, Eunsol Choi, Daniel S Weld, and Luke Zettlemoyer. 2017. Triviaqa: A large scale distantly supervised challenge dataset for reading comprehension. In *ACL*.
- Ryan Kiros, Yukun Zhu, Ruslan R Salakhutdinov, Richard Zemel, Raquel Urtasun, Antonio Torralba, and Sanja Fidler. 2015. Skip-thought vectors. In *Advances in neural information processing systems*, pages 3294–3302.
- Quoc Le and Tomas Mikolov. 2014. Distributed representations of sentences and documents. In *International Conference on Machine Learning*, pages 1188–1196.
- Hector J Levesque, Ernest Davis, and Leora Morgenstern. 2011. The winograd schema challenge. In *Aaai spring symposium: Logical formalizations of commonsense reasoning*, volume 46, page 47.
- Lajanugen Logeswaran and Honglak Lee. 2018. [An efficient framework for learning sentence representations](#). In *International Conference on Learning Representations*.
- Bryan McCann, James Bradbury, Caiming Xiong, and Richard Socher. 2017. Learned in translation: Contextualized word vectors. In *NIPS*.
- Oren Melamud, Jacob Goldberger, and Ido Dagan. 2016. context2vec: Learning generic context embedding with bidirectional LSTM. In *CoNLL*.
- Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S Corrado, and Jeff Dean. 2013. Distributed representations of words and phrases and their compositionality. In *Advances in Neural Information Processing Systems 26*, pages 3111–3119. Curran Associates, Inc.
- Andriy Mnih and Geoffrey E Hinton. 2009. [A scalable hierarchical distributed language model](#). In D. Koller, D. Schuurmans, Y. Bengio, and L. Bottou, editors, *Advances in Neural Information Processing Systems 21*, pages 1081–1088. Curran Associates, Inc.
- Ankur P Parikh, Oscar Täckström, Dipanjan Das, and Jakob Uszkoreit. 2016. A decomposable attention model for natural language inference. In *EMNLP*.
- Jeffrey Pennington, Richard Socher, and Christopher D. Manning. 2014. [Glove: Global vectors for word representation](#). In *Empirical Methods in Natural Language Processing (EMNLP)*, pages 1532–1543.
- Matthew Peters, Waleed Ammar, Chandra Bhagavatula, and Russell Power. 2017. Semi-supervised sequence tagging with bidirectional language models. In *ACL*.
- Matthew Peters, Mark Neumann, Mohit Iyyer, Matt Gardner, Christopher Clark, Kenton Lee, and Luke Zettlemoyer. 2018a. Deep contextualized word representations. In *NAACL*.
- Matthew Peters, Mark Neumann, Luke Zettlemoyer, and Wen-tau Yih. 2018b. Dissecting contextual word embeddings: Architecture and representation. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 1499–1509.
- Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. 2018. Improving language understanding with unsupervised learning. Technical report, OpenAI.
- Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. 2016. Squad: 100,000+ questions for machine comprehension of text. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 2383–2392.
- Minjoon Seo, Aniruddha Kembhavi, Ali Farhadi, and Hannaneh Hajishirzi. 2017. Bidirectional attention flow for machine comprehension. In *ICLR*.
- Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D Manning, Andrew Ng, and Christopher Potts. 2013. Recursive deep models for semantic compositionality over a sentiment treebank. In *Proceedings of the 2013 conference on empirical methods in natural language processing*, pages 1631–1642.
- Fu Sun, Linyang Li, Xipeng Qiu, and Yang Liu. 2018. U-net: Machine reading comprehension with unanswerable questions. *arXiv preprint arXiv:1810.06638*.
- Wilson L Taylor. 1953. Cloze procedure: A new tool for measuring readability. *Journalism Bulletin*, 30(4):415–433.
- Erik F Tjong Kim Sang and Fien De Meulder. 2003. Introduction to the conll-2003 shared task: Language-independent named entity recognition. In *CoNLL*.
- Joseph Turian, Lev Ratinov, and Yoshua Bengio. 2010. Word representations: A simple and general method for semi-supervised learning. In *Proceedings of the 48th Annual Meeting of the Association for Computational Linguistics, ACL ’10*, pages 384–394.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in Neural Information Processing Systems*, pages 6000–6010.
- Pascal Vincent, Hugo Larochelle, Yoshua Bengio, and Pierre-Antoine Manzagol. 2008. Extracting and composing robust features with denoising autoencoders. In *Proceedings of the 25th international conference on Machine learning*, pages 1096–1103. ACM.
- Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel Bowman. 2018a. Glue: A multi-task benchmark and analysis platform

曼达尔·乔希、尤恩索尔·崔、丹尼尔·S·韦尔德和卢克·泽特勒迈尔。2017。TriviaQA: 一个大规模远距离监督的阅读挑战数据集。载于《计算语言学协会年会》(ACL)

瑞安·基罗斯、朱颖坤、鲁斯兰·R·萨拉胡丁诺夫、理查德·泽梅尔、拉奎尔·乌拉塔苏恩、安东尼奥·托拉尔巴和桑贾·菲德勒。2015。跳跃思想向量。载于《神经信息处理系统进展》，第3294–3302页。

夸克·李和托马斯·米科洛夫。2014。句子与文档的分布式表示。载于《国际机器学习大会论文集》，第1188–1196页。

赫克托·J·勒维斯克、欧内斯特·戴维斯和莉奥拉·摩根斯特恩。2011。威诺格拉德模式挑战。载于《会议论文集》AAAI春季研讨会：逻辑形式化常识推理，第46卷，第47页。拉贾努根·洛格斯瓦

兰和洪拉克·李。2018。一种学习句子表示的高效框架。载于《国际学习表征会议》

布莱恩·麦卡恩、詹姆斯·布拉德伯里、蔡铭雄和理查德·索彻。2017。翻译中习得：上下文文化向量。载于《神经信息处理系统大会》(NIPS)

奥伦·梅拉穆德、雅各布·戈德伯格和伊多·达甘。2016。context2vec: 利用双向LSTM学习通用上下文嵌入。载于《计算语言学会议》(CoNLL)。托马斯·

米科洛夫、伊利亚·苏茨克维、凯·陈、格雷格·S·科尔。拉多和杰夫·迪恩。2013。词与短语的分布式表示及其组合性。载于《神经信息处理系统进展26》，第3111–3119页。Curran Associates, Inc.

安德里·米尼赫和杰弗里·E·欣顿。2009。一种可扩展的分层分布式语言模型。载于D. 科勒、D. 舒尔曼斯、Y. 本吉奥和L. 博图编，《神经信息处理系统进展21》，第1081–1088页。Curran Associates, Inc.

安库尔·P·帕里克、奥斯卡·泰克斯特罗姆、迪潘简·达斯和雅各布·乌斯佐雷特。2016。一种可分解注意力模型，用于自然语言推理。载于《经验方法在自然语言处理中的应用》(EMNLP)。

杰弗里·彭宁顿、理查德·索彻和克里斯托弗·D·曼宁。克里斯托弗·D·曼宁。2014。GloVe: 词表示的全局向量。载于《自然语言处理经验方法》(EMNLP)，第1532–1543页。

马修·彼得斯、韦莱德·阿玛尔、钱德拉·巴格瓦图拉和拉塞尔·鲍尔。2017。利用双向语言模型进行半监督序列标注。载于《计算语言学协会年会》(ACL)

马修·彼得斯、马克·纽曼、莫希特·伊耶尔、马特·加德纳、克里斯托弗·克拉克、肯顿·李和卢克·泽特勒迈尔。2018a。深度上下文文化词表示。载于《北美计算语言学协会年会》(NAACL)

马修·彼得斯、马克·纽曼、卢克·泽特勒迈尔和温涛·易。2018b。剖析上下文词嵌入：架构与表征。载于《2018年自然语言处理经验方法会议论文集》，第1499–1509页。

亚历克·拉德福德、卡西克·纳拉西姆汉、蒂姆·萨利曼斯和伊利亚·苏茨克维。2018。借助无监督学习提升语言理解能力。技术报告，OpenAI

普拉纳夫·拉朱帕卡尔、张健、康斯坦丁·洛皮列夫和珀西·梁。2016。SQuAD: 面向文本机器理解的10万+问题集。载于《会议论文集》2016年自然语言处理经验方法会议，自然语言处理，第2383–2392页。闵俊均、阿尼

鲁达·肯巴维、阿里·法尔哈迪和汉娜·哈吉希尔斯。2017。用于机器理解的双向注意力流。载于《国际学习表征会议》(ICLR)

理查德·索彻、亚历克斯·佩雷林、珍·吴、杰森·朱安、克里斯托弗·D·曼宁、安德鲁·吴和克里斯托弗·波特。2013。用于情感树库语义组合性的递归深度模型。载于《2013年自然语言处理经验方法会议论文集》，第1631–1642页。

孙富、李林阳、邱锡鹏和刘洋。2018。U-Net: 面向不可回答问题的机器阅读理解模型。arXiv预印本 arXiv:1810.06638

威尔逊·L·泰勒。1953。完形填空程序：一种衡量可读性的新工具。《新闻学公报》，30(4): 415–433。

埃里克·F·通琼·金·桑和菲恩·德·梅乌尔德。2003。CoNLL-2003共享任务导论：与语言无关的命名实体识别。载于《计算语言学会议》(CoNLL)

约瑟夫·图里安、列夫·拉蒂诺夫和约书亚·本吉奥。2010。词表示：一种简单通用的半监督学习方法。载于《会议论文集》第48届计算语言学协会年会，计算语言学协会年会ACL 10，第384–394页。阿希

什·瓦斯瓦尼、诺姆·沙泽尔、尼基·帕尔马、雅各布·乌斯佐雷特、利昂·琼斯、艾丹·N·戈麦斯、卢卡·什·凯撒和伊利亚·波洛苏欣。2017。注意力就是你所需要的全部。载于《神经信息处理系统进展》，第6000–6010页。

帕斯卡尔·文森特、雨果·拉罗谢尔、约书亚·本吉奥和皮埃尔-安托万·曼扎戈尔。2008。利用去噪自编码器提取并组合鲁棒特征。载于《第25届国际机器学习大会论文集》，第1096–1103页。ACM出版社。

亚历克斯·王、阿曼普里特·辛格、朱利安·迈克尔、费利克斯·希尔、奥默·列维和塞缪尔·鲍曼。2018a。GLUE: 一个多任务基准与分析平台

for natural language understanding. In *Proceedings of the 2018 EMNLP Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP*, pages 353–355.

Wei Wang, Ming Yan, and Chen Wu. 2018b. Multi-granularity hierarchical attention fusion networks for reading comprehension and question answering. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics.

Alex Warstadt, Amanpreet Singh, and Samuel R Bowman. 2018. Neural network acceptability judgments. *arXiv preprint arXiv:1805.12471*.

Adina Williams, Nikita Nangia, and Samuel R Bowman. 2018. A broad-coverage challenge corpus for sentence understanding through inference. In *NAACL*.

Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, et al. 2016. Google’s neural machine translation system: Bridging the gap between human and machine translation. *arXiv preprint arXiv:1609.08144*.

Jason Yosinski, Jeff Clune, Yoshua Bengio, and Hod Lipson. 2014. How transferable are features in deep neural networks? In *Advances in neural information processing systems*, pages 3320–3328.

Adams Wei Yu, David Dohan, Minh-Thang Luong, Rui Zhao, Kai Chen, Mohammad Norouzi, and Quoc V Le. 2018. QANet: Combining local convolution with global self-attention for reading comprehension. In *ICLR*.

Rowan Zellers, Yonatan Bisk, Roy Schwartz, and Yejin Choi. 2018. Swag: A large-scale adversarial dataset for grounded commonsense inference. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Yukun Zhu, Ryan Kiros, Rich Zemel, Ruslan Salakhutdinov, Raquel Urtasun, Antonio Torralba, and Sanja Fidler. 2015. Aligning books and movies: Towards story-like visual explanations by watching movies and reading books. In *Proceedings of the IEEE international conference on computer vision*, pages 19–27.

Appendix for “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”

We organize the appendix into three sections:

- Additional implementation details for BERT are presented in Appendix A;

- Additional details for our experiments are presented in Appendix B; and

- Additional ablation studies are presented in Appendix C.

We present additional ablation studies for BERT including:

- Effect of Number of Training Steps; and
- Ablation for Different Masking Procedures.

A Additional Details for BERT

A.1 Illustration of the Pre-training Tasks

We provide examples of the pre-training tasks in the following.

Masked LM and the Masking Procedure Assuming the unlabeled sentence is *my dog is hairy*, and during the random masking procedure we chose the 4-th token (which corresponding to *hairy*), our masking procedure can be further illustrated by

- 80% of the time: Replace the word with the [MASK] token, e.g., *my dog is hairy* → *my dog is [MASK]*
- 10% of the time: Replace the word with a random word, e.g., *my dog is hairy* → *my dog is apple*
- 10% of the time: Keep the word unchanged, e.g., *my dog is hairy* → *my dog is hairy*. The purpose of this is to bias the representation towards the actual observed word.

The advantage of this procedure is that the Transformer encoder does not know which words it will be asked to predict or which have been replaced by random words, so it is forced to keep a distributional contextual representation of every input token. Additionally, because random replacement only occurs for 1.5% of all tokens (i.e., 10% of 15%), this does not seem to harm the model’s language understanding capability. In Section C.2, we evaluate the impact this procedure.

Compared to standard language model training, the masked LM only make predictions on 15% of tokens in each batch, which suggests that more pre-training steps may be required for the model

自然语言理解。载于《会议论文集》

2018年EMNLP研讨会BlackboxNLP：自然语言处理神经网络的分析与解读，自然语言处理神经网络的分析与解读，第353-355页。

魏伟、闫明、吴晨。2018b。多粒度分层注意力融合网络用于阅读理解与问答。载于《第56届计算语言学协会年会论文集》

计算语言学协会（第一卷：长论文）。计算语言学协会。

亚历克斯·瓦尔斯塔特、阿曼普里特·辛格和塞缪尔·R·鲍曼。2018。神经网络可接受性判断。arXiv预印本 arXiv:1805.12471

阿迪娜·威廉姆斯、尼基塔·南吉亚和塞缪尔·R·鲍曼。2018。一个覆盖范围广泛的挑战语料库，用于通过推理实现句子理解。载于《北美计算语言学协会年会》（NAACL）

雍辉·吴、迈克·舒斯特、志峰·陈、夸克·V·李、穆罕默德·诺鲁齐、沃尔夫冈·马赫雷、马克西姆·克里昆、袁·曹、秦·高、克劳斯·马赫雷等。2016。谷歌神经机器翻译系统：弥合人类翻译与机器翻译之间的鸿沟。arXiv预印本 arXiv:1609.08144

杰森·约辛斯基、杰夫·克鲁恩、约书亚·本吉奥和霍德利普森。2014。深度神经网络中的特征可迁移性如何？载于《神经信息处理系统进展》，第3320-3328页

亚当斯·魏·于、大卫·多汉、明·唐·卢翁、瑞·赵、凯·陈、穆罕默德·诺鲁齐和夸克·V·李。2018。QANet：结合局部卷积与全局自注意力机制的阅读理解模型。载于《国际学习表征会议》（ICLR）

罗文·泽勒斯、约纳坦·比斯克、罗伊·施瓦茨和叶瑾崔。2018。SWAG：一个大规模对抗性数据集，用于基于场景的常识推理。载于《会议论文集》2018年经验方法会议论文集自然语言处理经验方法会议（EMNLP）。Yukun

Zhu, Ryan Kiros, Rich Zemel, Ruslan Salakhutdinov, Raquel Urtasun, Antonio Torralba 和 Sanja Fidler。2015。对齐图书与电影：通过观看电影和阅读图书生成类故事的视觉解释。载于IEEE国际计算机视觉会议论文集，第19-27页。

BERT附录：面向语言理解的深度双向Transformer预训练语言理解

我们将附录分为三个部分：

- BERT的更多实现细节见附录A；

- 实验的更多细节见附录B；以及

- 更多消融研究见附录C

我们展示了针对BERT的额外消融研究，包括：

- 训练步数数量的影响；以及 - 不同掩码流程的消融分析流程

A BERT附加细节

A.1 预训练任务示意图

以下提供了预训练任务的示例

掩码语言建模（Masked LM）及掩码流程：假设未标注句子为my dog is hairy，在随机掩码流程中我们选中第4个词元（对应hairy），则该掩码流程可进一步示意为

- 80%的情况下：将该词替换为[MASK]标记，例如：my dog is hairy →

my dog is [MASK]

- 10%的情况下：将该词替换为一个随机词，例如：my dog is hairy → my

dog is apple

- 10%的情况下：保留原词不变
保持不变，例如：my dog is hairy → my dog is hairy。这样做的目的是使表征偏向实际观测到的词

该流程的优势在于：Transformer编码器无法预知哪些词元将被要求预测，也无法预知哪些词元已被随机词元替换，因此它必须为每个输入词元维持一种分布式的上下文表征。此外，由于随机替换仅发生在全部词元的1.5%（即15%中的10%）上，这似乎并未损害模型的语言理解能力。我们在C.2节中评估了该流程的影响

与标准语言模型训练相比，掩码语言建模（Masked LM）仅对每个批次中15%的词元进行预测，这意味着模型可能需要更多的预训练步数

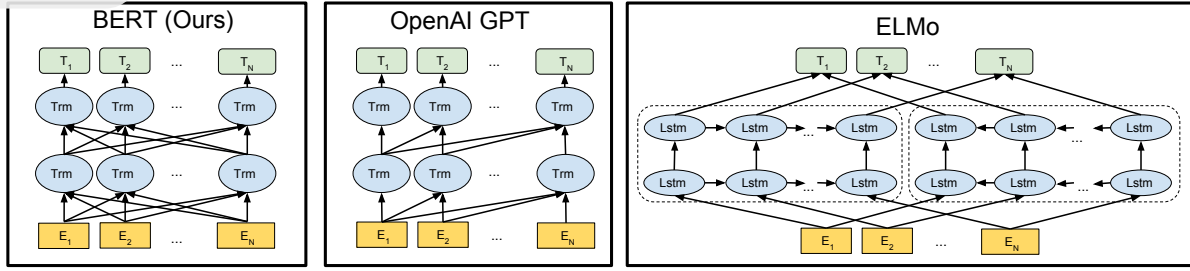


Figure 3: Differences in pre-training model architectures. BERT uses a bidirectional Transformer. OpenAI GPT uses a left-to-right Transformer. ELMo uses the concatenation of independently trained left-to-right and right-to-left LSTMs to generate features for downstream tasks. Among the three, only BERT representations are jointly conditioned on both left and right context in all layers. In addition to the architecture differences, BERT and OpenAI GPT are fine-tuning approaches, while ELMo is a feature-based approach.

to converge. In Section C.1 we demonstrate that MLM does converge marginally slower than a left-to-right model (which predicts every token), but the empirical improvements of the MLM model far outweigh the increased training cost.

Next Sentence Prediction The next sentence prediction task can be illustrated in the following examples.

Input = [CLS] the man went to [MASK] store [SEP]
 he bought a gallon [MASK] milk [SEP]
 Label = IsNext

Input = [CLS] the man [MASK] to the store [SEP]
 penguin [MASK] are flight ##less birds [SEP]
 Label = NotNext

A.2 Pre-training Procedure

To generate each training input sequence, we sample two spans of text from the corpus, which we refer to as “sentences” even though they are typically much longer than single sentences (but can be shorter also). The first sentence receives the A embedding and the second receives the B embedding. 50% of the time B is the actual next sentence that follows A and 50% of the time it is a random sentence, which is done for the “next sentence prediction” task. They are sampled such that the combined length is ≤ 512 tokens. The LM masking is applied after WordPiece tokenization with a uniform masking rate of 15%, and no special consideration given to partial word pieces.

We train with batch size of 256 sequences (256 sequences * 512 tokens = 128,000 tokens/batch) for 1,000,000 steps, which is approximately 40

epochs over the 3.3 billion word corpus. We use Adam with learning rate of $1e-4$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, L2 weight decay of 0.01, learning rate warmup over the first 10,000 steps, and linear decay of the learning rate. We use a dropout probability of 0.1 on all layers. We use a `gelu` activation (Hendrycks and Gimpel, 2016) rather than the standard `relu`, following OpenAI GPT. The training loss is the sum of the mean masked LM likelihood and the mean next sentence prediction likelihood.

Training of BERT_{BASE} was performed on 4 Cloud TPUs in Pod configuration (16 TPU chips total).¹³ Training of BERT_{LARGE} was performed on 16 Cloud TPUs (64 TPU chips total). Each pre-training took 4 days to complete.

Longer sequences are disproportionately expensive because attention is quadratic to the sequence length. To speed up pretraining in our experiments, we pre-train the model with sequence length of 128 for 90% of the steps. Then, we train the rest 10% of the steps of sequence of 512 to learn the positional embeddings.

A.3 Fine-tuning Procedure

For fine-tuning, most model hyperparameters are the same as in pre-training, with the exception of the batch size, learning rate, and number of training epochs. The dropout probability was always kept at 0.1. The optimal hyperparameter values are task-specific, but we found the following range of possible values to work well across all tasks:

- **Batch size:** 16, 32

¹³<https://cloudplatform.googleblog.com/2018/06/Cloud-TPU-now-offers-preemptible-pricing-and-global-availability.html>

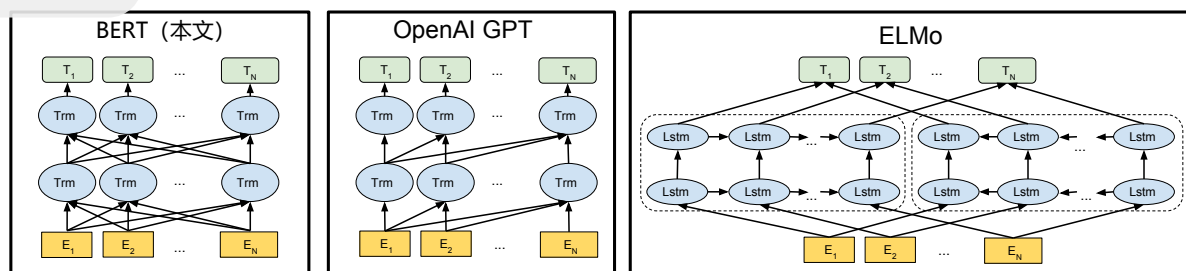


图3：预训练模型架构的差异。BERT使用双向Transformer；OpenAI GPT使用从左到右的Transformer；ELMo则通过拼接两个独立训练的、分别从左到右和从右到左的LSTM，为下游任务生成特征。三者之中，仅有BERT的表征在所有层中均同时依赖于左右上下文。除架构差异外，BERT和OpenAI GPT属于微调方法，而ELMo属于基于特征的方法。

以实现收敛。在附录C.1中，我们证明掩码语言建模（MLM）的收敛速度略慢于从左到右的语言模型（后者预测每个词元），但MLM模型带来的实证性能提升远超其增加的训练开销。

下一句预测 下一句预测任务可通过以下示例说明。

输入 = [CLS] the man went to [MASK] store [SEP] he bought a
gallon [MASK] milk [SEP]
标签 = IsNext

输入 = [CLS] the man [MASK] to the store [SEP] penguin [MASK]
are flight ##less birds [SEP] 标签 = NotNext

A.2 预训练流程

为生成每个训练输入序列，我们从语料库中采样两段文本，我们称之为“句子”，尽管它们通常远长于单句（但也可能更短）。第一句被赋予A嵌入，第二句被赋予B嵌入。其中50%的情况下，B确实是紧随A之后的实际下一句；另50%的情况下，B则是随机选取的句子——这一设定服务于下一句预测任务。采样时确保两段文本总长度不超过512个词元。掩码操作在WordPiece分词后执行，统一掩码率为15%，且未对子词片段作特殊处理。

我们以256个序列为一批（256序列 × 512词元 = 每批128,000词元）训练100万步，约相当于40轮。

在包含33亿词的语料库上训练若干轮。我们采用Adam优化器，学习率为 $1e-4$ ， $\beta = 0.9$ ， $\beta = 0.999$ ，L2权重衰减为0.01，前10,000步进行学习率预热，并在线性衰减学习率。所有层的Dropout概率均为0.1。我们采用GELU激活函数（Hendrycks和Gimpel, 2016），而非标准ReLU，此做法与OpenAI GPT一致。训练损失为平均掩码语言建模似然与平均下一句预测似然之和。

BERTBASE的训练在Pod配置下的4块Cloud TPU（共16个TPU芯片）上完成。¹³ BERTLARGE的训练则在16块Cloud TPU（共64个TPU芯片）上完成。每次预训练耗时约4天。

由于注意力计算复杂度与序列长度呈平方关系，较长序列的计算开销显著更高。为加快实验中的预训练速度，我们对90%的训练步数采用长度为128的序列进行预训练；随后，对剩余10%的训练步数采用长度为512的序列进行训练，以学习位置嵌入。

A.3 微调流程

微调时，大多数模型超参数与预训练阶段保持一致，仅批量大小、学习率和训练轮数例外。所有层的Dropout概率始终设为0.1。最优超参数值因任务而异，但我们发现以下取值范围在所有任务上均表现良好：

- 批量大小：16、32

¹³<https://cloudplatform.googleblog.com/2018/06/Cloud-TPU-now-offers-preemptible-pricing-and-global-availability.html>

- **Learning rate (Adam):** 5e-5, 3e-5, 2e-5
- **Number of epochs:** 2, 3, 4

We also observed that large data sets (e.g., 100k+ labeled training examples) were far less sensitive to hyperparameter choice than small data sets. Fine-tuning is typically very fast, so it is reasonable to simply run an exhaustive search over the above parameters and choose the model that performs best on the development set.

A.4 Comparison of BERT, ELMo, and OpenAI GPT

Here we study the differences in recent popular representation learning models including ELMo, OpenAI GPT and BERT. The comparisons between the model architectures are shown visually in Figure 3. Note that in addition to the architecture differences, BERT and OpenAI GPT are fine-tuning approaches, while ELMo is a feature-based approach.

The most comparable existing pre-training method to BERT is OpenAI GPT, which trains a left-to-right Transformer LM on a large text corpus. In fact, many of the design decisions in BERT were intentionally made to make it as close to GPT as possible so that the two methods could be minimally compared. The core argument of this work is that the bi-directionality and the two pre-training tasks presented in Section 3.1 account for the majority of the empirical improvements, but we do note that there are several other differences between how BERT and GPT were trained:

- GPT is trained on the BooksCorpus (800M words); BERT is trained on the BooksCorpus (800M words) and Wikipedia (2,500M words).
- GPT uses a sentence separator ([SEP]) and classifier token ([CLS]) which are only introduced at fine-tuning time; BERT learns [SEP], [CLS] and sentence A/B embeddings during pre-training.
- GPT was trained for 1M steps with a batch size of 32,000 words; BERT was trained for 1M steps with a batch size of 128,000 words.
- GPT used the same learning rate of 5e-5 for all fine-tuning experiments; BERT chooses a task-specific fine-tuning learning rate which performs the best on the development set.

To isolate the effect of these differences, we perform ablation experiments in Section 5.1 which demonstrate that the majority of the improvements are in fact coming from the two pre-training tasks and the bidirectionality they enable.

A.5 Illustrations of Fine-tuning on Different Tasks

The illustration of fine-tuning BERT on different tasks can be seen in Figure 4. Our task-specific models are formed by incorporating BERT with one additional output layer, so a minimal number of parameters need to be learned from scratch. Among the tasks, (a) and (b) are sequence-level tasks while (c) and (d) are token-level tasks. In the figure, E represents the input embedding, T_i represents the contextual representation of token i , [CLS] is the special symbol for classification output, and [SEP] is the special symbol to separate non-consecutive token sequences.

B Detailed Experimental Setup

B.1 Detailed Descriptions for the GLUE Benchmark Experiments.

Our GLUE results in Table 1 are obtained from <https://gluebenchmark.com/leaderboard> and <https://blog.openai.com/language-unsupervised>. The GLUE benchmark includes the following datasets, the descriptions of which were originally summarized in Wang et al. (2018a):

MNLI Multi-Genre Natural Language Inference is a large-scale, crowdsourced entailment classification task (Williams et al., 2018). Given a pair of sentences, the goal is to predict whether the second sentence is an *entailment*, *contradiction*, or *neutral* with respect to the first one.

QQP Quora Question Pairs is a binary classification task where the goal is to determine if two questions asked on Quora are semantically equivalent (Chen et al., 2018).

QNLI Question Natural Language Inference is a version of the Stanford Question Answering Dataset (Rajpurkar et al., 2016) which has been converted to a binary classification task (Wang et al., 2018a). The positive examples are (question, sentence) pairs which do contain the correct answer, and the negative examples are (question, sentence) from the same paragraph which do not contain the answer.

- 学习率 (Adam优化器) : 5e-5、3e-5、2e-5
- 训练轮数: 2、3、4

我们还观察到, 大规模数据集 (例如含10万以上标注训练样本的数据集) 对超参数选择的敏感性远低于小规模数据集。微调通常非常快速, 因此对上述超参数进行穷举搜索, 并选择在开发集上表现最佳的模型, 是一种合理做法。

A.4 BERT、ELMo与OpenAI GPT的对比

本文探讨了近期主流表征学习模型 (包括ELMo、OpenAI GPT和BERT) 之间的差异。各模型架构的对比在图3中以可视化方式呈现。需要注意的是, 除架构差异外, BERT和OpenAI GPT属于微调方法, 而ELMo属于基于特征的方法。

目前与BERT最接近的现有预训练方法是OpenAI GPT, 它在大规模文本语料库上训练一个从左到右的Transformer语言模型。事实上, BERT中的许多设计决策都是有意为之, 旨在使其尽可能接近GPT, 从而实现二者之间的最小化对比。本文的核心论点是: 第3.1节提出的双向性及两项预训练任务解释了绝大部分实证性能提升; 但我们也注意到, BERT与GPT在训练方式上尚存在其他若干差异:

- GPT在BooksCorpus语料库 (8亿词) 上训练; BERT则在BooksCorpus语料库 (8亿词) 和Wikipedia语料库 (25亿词) 上联合训练。
- GPT使用的句子分隔符 ([SEP]) 和分类器词元 ([CLS]) 仅在微调阶段引入; 而BERT在预训练阶段即已学习[SEP]、[CLS]以及句子A/B嵌入。
- GPT在词数批量大小为32,000的情况下训练了100万步; BERT在词数批量大小为128,000的情况下同样训练了100万步。
- GPT在所有微调实验中均采用相同的学习率5e-5; BERT则针对不同任务选择在开发集上表现最佳的微调学习率。

为单独考察这些差异的影响, 我们在第5.1节中进行了消融实验, 结果表明, 大部分性能提升实际上源自第3.1节所介绍的两项预训练任务及其所支持的双向性。

A.5 在不同任务上的微调示意图

图4展示了在不同任务上对BERT进行微调的示意图。我们的任务特定模型通过将BERT与一个额外的输出层相结合而构建, 因此仅需从头学习极少数量的参数。在这些任务中, (a)和(b)属于序列级任务, 而(c)和(d)属于词元级任务。图中, E表示输入嵌入, T_i 表示第*i*个词元的上下文表示, [CLS]是用于分类输出的特殊符号, [SEP]是用于分隔非连续词元序列的特殊符号。

B 详细实验设置

B.1 GLUE 基准测试实验的详细说明。

表 1 中我们的 GLUE 结果取自 <https://gluebenchmark.com/leaderboard> 和 <https://blog.openai.com/language-unsupervised>。GLUE 基准测试包含以下数据集, 其描述最初由 Wang 等 (2018a) 归纳整理:

MNLI 多领域自然语言推理是一个大规模众包的蕴含分类任务 (Williams 等, 2018)。给定一对句子, 目标是预测第二句相对于第一句属于蕴含、矛盾还是中立关系。

QQP Quora 问题对是一个二分类任务, 目标是判断在 Quora 上提出的两个问题在语义上是否等价 (Chen 等, 2018)。

QNLI 问题自然语言推理是斯坦福问答数据集 (Rajpurkar 等, 2016) 的一个变体, 已被转换为二分类任务 (Wang 等, 2018a)。正样本是包含正确答案的 (问题, 句子) 对; 负样本则是来自同一段落但不包含答案的 (问题, 句子) 对。

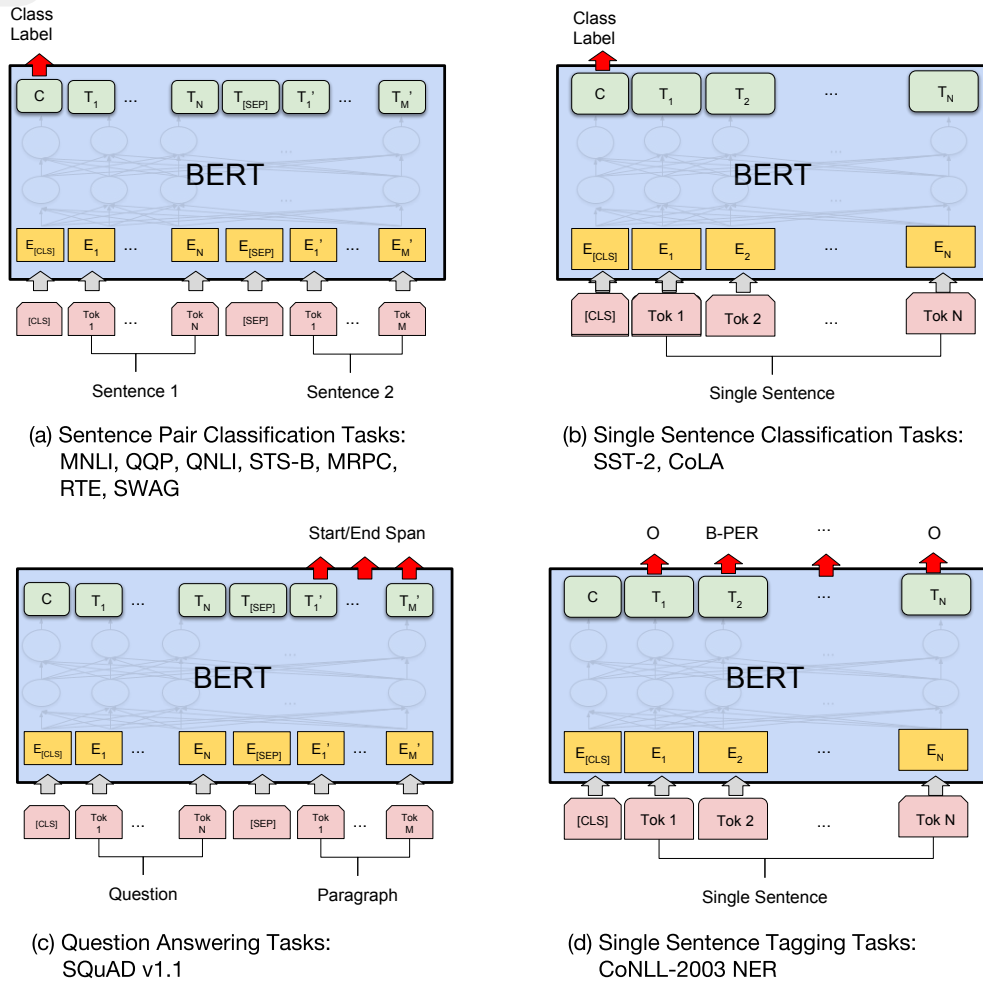


Figure 4: Illustrations of Fine-tuning BERT on Different Tasks.

SST-2 The Stanford Sentiment Treebank is a binary single-sentence classification task consisting of sentences extracted from movie reviews with human annotations of their sentiment (Socher et al., 2013).

CoLA The Corpus of Linguistic Acceptability is a binary single-sentence classification task, where the goal is to predict whether an English sentence is linguistically “acceptable” or not (Warstadt et al., 2018).

STS-B The Semantic Textual Similarity Benchmark is a collection of sentence pairs drawn from news headlines and other sources (Cer et al., 2017). They were annotated with a score from 1 to 5 denoting how similar the two sentences are in terms of semantic meaning.

MRPC Microsoft Research Paraphrase Corpus consists of sentence pairs automatically extracted from online news sources, with human annotations

for whether the sentences in the pair are semantically equivalent (Dolan and Brockett, 2005).

RTE Recognizing Textual Entailment is a binary entailment task similar to MNLI, but with much less training data (Bentivogli et al., 2009).¹⁴

WNLI Winograd NLI is a small natural language inference dataset (Levesque et al., 2011). The GLUE webpage notes that there are issues with the construction of this dataset,¹⁵ and every trained system that’s been submitted to GLUE has performed worse than the 65.1 baseline accuracy of predicting the majority class. We therefore exclude this set to be fair to OpenAI GPT. For our GLUE submission, we always predicted the ma-

¹⁴Note that we only report single-task fine-tuning results in this paper. A multitask fine-tuning approach could potentially push the performance even further. For example, we did observe substantial improvements on RTE from multitask training with MNLI.

¹⁵<https://gluebenchmark.com/faq>

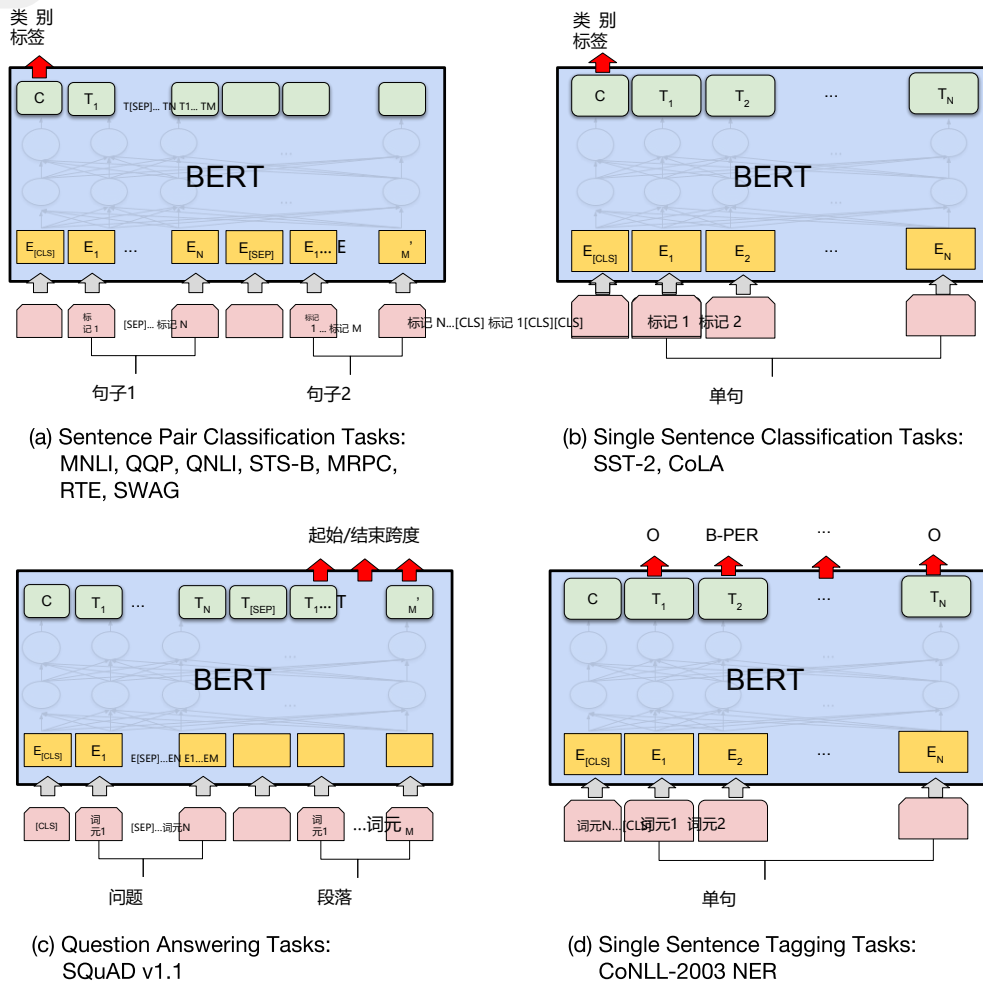


图4：BERT在不同任务上的微调示意图。

SST-2 (斯坦福情感树库) 是一项二元单句分类任务，包含从影评中提取的句子，并附有人工标注的情感极性 (Socher等, 2013)。

CoLA (语言可接受性语料库) 是一项二元单句分类任务，目标是判断一个英文句子在语言学上是否可接受 (Warstadt等, 2018)。

STS-B (语义文本相似度基准) 是一组从新闻标题及其他来源采集的句子对 (Cer等, 2017)。每对句子均标有1至5分的相似度评分，表示二者在语义意义上的相似程度。

MRPC (微软研究院释义语料库) 由从在线新闻源自动抽取的句子对构成，并附有人工标注。

RTE (文本蕴含识别) 是一项二元蕴含任务，与MNLI类似，但训练数据少得多

(Bentivogli等, 2009)¹⁴;
WNLI (Winograd NLI) 是一个小型自然语言推理数据集 (Levesque等, 2011)。

GLUE官网指出该数据集在构建上存在若干问题¹⁵，且所有提交至GLUE的已训练系统在此数据集上的表现均低于65.1%这一多数类预测基线准确率。因此，为公平起见，我们未将该数据集纳入与OpenAI GPT的对比。在我们的GLUE提交中，我们始终预测多数类。

¹⁴ 注意，本文仅报告单任务微调结果。多任务微调方法可能进一步提升性能。例如，我们观察到在MNLI上进行多任务训练可显著提升RTE任务的表现。

¹⁵ <https://gluebenchmark.com/faq>

majority class.

C Additional Ablation Studies

C.1 Effect of Number of Training Steps

Figure 5 presents MNLI Dev accuracy after fine-tuning from a checkpoint that has been pre-trained for k steps. This allows us to answer the following questions:

1. Question: Does BERT really need such a large amount of pre-training (128,000 words/batch * 1,000,000 steps) to achieve high fine-tuning accuracy?

Answer: Yes, BERT_{BASE} achieves almost 1.0% additional accuracy on MNLI when trained on 1M steps compared to 500k steps.

2. Question: Does MLM pre-training converge slower than LTR pre-training, since only 15% of words are predicted in each batch rather than every word?

Answer: The MLM model does converge slightly slower than the LTR model. However, in terms of absolute accuracy the MLM model begins to outperform the LTR model almost immediately.

C.2 Ablation for Different Masking Procedures

In Section 3.1, we mention that BERT uses a mixed strategy for masking the target tokens when pre-training with the masked language model (MLM) objective. The following is an ablation study to evaluate the effect of different masking strategies.

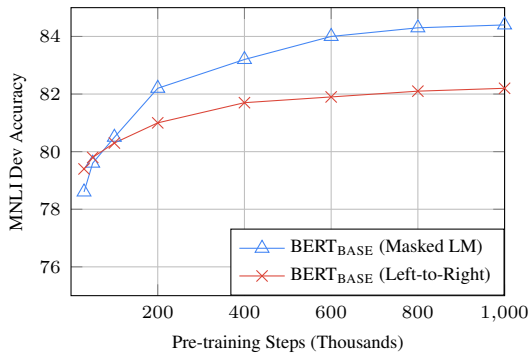


Figure 5: Ablation over number of training steps. This shows the MNLI accuracy after fine-tuning, starting from model parameters that have been pre-trained for k steps. The x-axis is the value of k .

Note that the purpose of the masking strategies is to reduce the mismatch between pre-training and fine-tuning, as the [MASK] symbol never appears during the fine-tuning stage. We report the Dev results for both MNLI and NER. For NER, we report both fine-tuning and feature-based approaches, as we expect the mismatch will be amplified for the feature-based approach as the model will not have the chance to adjust the representations.

Masking Rates			Dev Set Results		
MASK	SAME	RND	MNLI	NER	
			Fine-tune	Fine-tune	Feature-based
80%	10%	10%	84.2	95.4	94.9
100%	0%	0%	84.3	94.9	94.0
80%	0%	20%	84.1	95.2	94.6
80%	20%	0%	84.4	95.2	94.7
0%	20%	80%	83.7	94.8	94.6
0%	0%	100%	83.6	94.9	94.6

Table 8: Ablation over different masking strategies.

The results are presented in Table 8. In the table, MASK means that we replace the target token with the [MASK] symbol for MLM; SAME means that we keep the target token as is; RND means that we replace the target token with another random token.

The numbers in the left part of the table represent the probabilities of the specific strategies used during MLM pre-training (BERT uses 80%, 10%, 10%). The right part of the paper represents the Dev set results. For the feature-based approach, we concatenate the last 4 layers of BERT as the features, which was shown to be the best approach in Section 5.3.

From the table it can be seen that fine-tuning is surprisingly robust to different masking strategies. However, as expected, using only the MASK strategy was problematic when applying the feature-based approach to NER. Interestingly, using only the RND strategy performs much worse than our strategy as well.

多数类。

C 补充消融实验

C.1 训练步数的影响

图5展示了从已预训练k步的检查点出发、经微调后在MNLI开发集上取得的准确率。这使我们能够回答以下问题：

1. 问题：BERT是否真需要如此大规模的预训练（每批128,000个词 × 100万步），才能实现较高的微调准确率？

答：是的，相较于训练50万步，BERTBASE在训练100万步后在MNLI上的准确率提升了近1.0%。

2. 问题：由于每个批次中仅有15%的词被预测，而非全部词，MLM预训练是否比LTR预训练收敛更慢？

答：MLM模型的收敛速度确实略慢于LTR模型。然而，就绝对准确率而言，MLM模型几乎从一开始就优于LTR模型。

C.2 不同掩码流程的消融实验

在第3.1节中，我们提到BERT在使用掩码语言建模（MLM）目标进行预训练时，对目标词元采用混合掩码策略。以下是一项消融研究，用于评估不同掩码策略的效果。

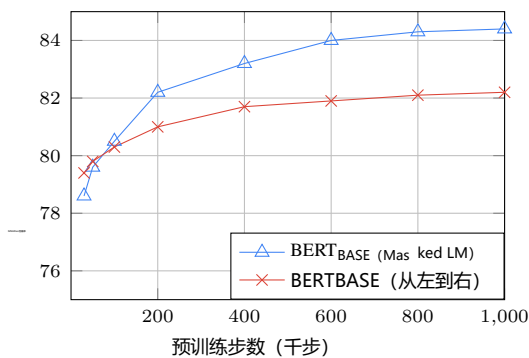


图5：关于训练步数的消融实验。该图显示了从已预训练k步的模型参数开始微调后，在MNLI任务上取得的准确率。横轴为k值。

需要注意的是，掩码策略的目的是减少预训练与微调之间的不匹配，因为在微调阶段永远不会出现[MASK]符号。我们报告了MNLI和NER的开发集结果。对于NER，我们同时报告了微调方法和基于特征的方法的结果，因为我们预期在基于特征的方法中这种不匹配会被放大，因为模型将无法调整其表征。

掩码率			开发集结果		
MASK	SAME	RND	MNLI 微调	NER 基于特征	NER 微调
80%	10%	10%	84.2	95.4	94.9
100%	0%	0%	84.3	94.9	94.0
80%	0%	20%	84.1	95.2	94.6
80%	20%	0%	84.4	95.2	94.7
0%	20%	80%	83.7	94.8	94.6
0%	0%	100%	83.6	94.9	94.6

表8：不同掩码策略的消融实验。

结果见表8。表中，MASK表示在MLM中将目标词元替换为[MASK]符号；SAME表示保持目标词元不变；RND表示将目标词元替换为另一个随机词元。

表左侧的数字代表MLM预训练期间各策略所使用的概率（BERT采用80%、10%、10%）。论文右侧部分展示的是开发集（Dev set）结果。对于基于特征的方法，我们将BERT最后4层的输出拼接作为特征，该方法在第5.3节中被证明效果最佳。

从表中可以看出，微调对不同掩码策略表现出人意料的鲁棒性。然而，正如预期，当在命名实体识别（NER）任务中采用基于特征的方法时，仅使用MASK策略存在问题。有趣的是，仅使用RND策略的效果也远逊于我们的策略。